Advanced CSS: Layout

Chapter 5

Randy Connolly and Ricardo Hoar

© 2015 Pearson

Objectives

1. Normal Flow
2. Positioning Elements
3. Floating Elements
4. Multicolumn Layouts
5. Approaches to CSS Layout
6. Responsive Design
7. CSS Frameworks
8. CSS Animations

Section 1 of 7

NORMAL FLOW

Normal Flow

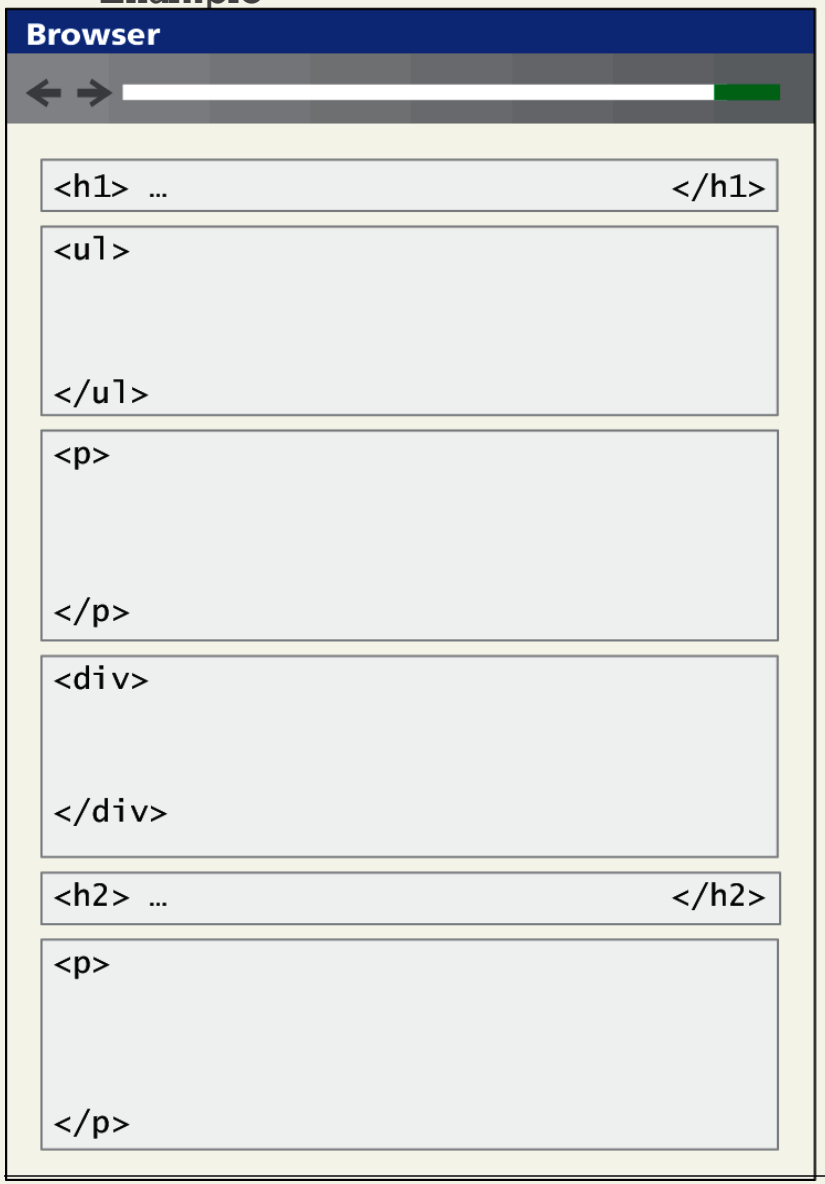
If it's so normal why have I never heard of it?

Normal flow refers here to how the browser will normally display block-level elements and inline elements from left to right and from top to bottom

- **Block-level elements** are each contained on their own line
  - <p>, , <h2>, <ul>, <table>, ...
- **Inline elements** do not form their own blocks but instead are displayed within lines
  - , <a>, <img>, , ...

Normal Flow

Example



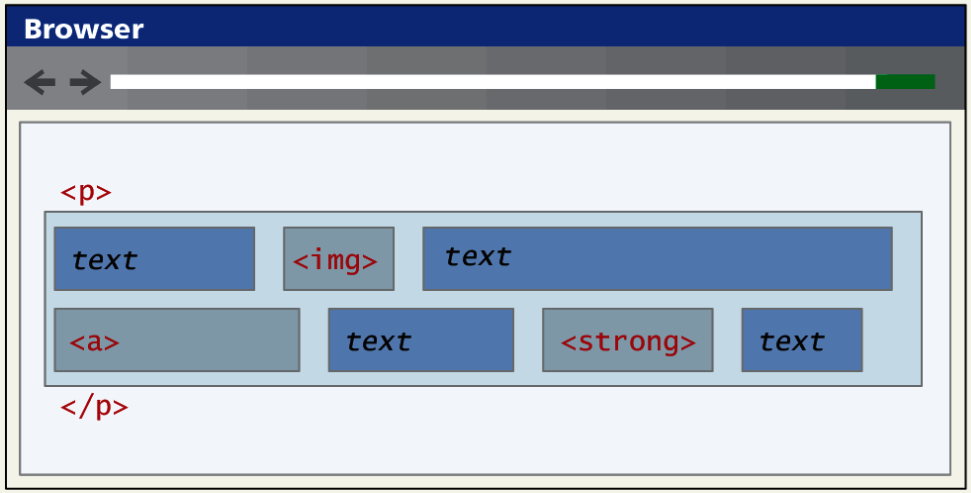
Each block exists on its own line and is displayed in normal flow from the browser window's top to its bottom.

By default each block-level element fills up the entire width of its parent (in this case, it is the <body>, which is equivalent to the width of the browser window).

You can use CSS box model properties to customize, for instance, the width of the box and the margin space between other block-level elements.

Normal Flow

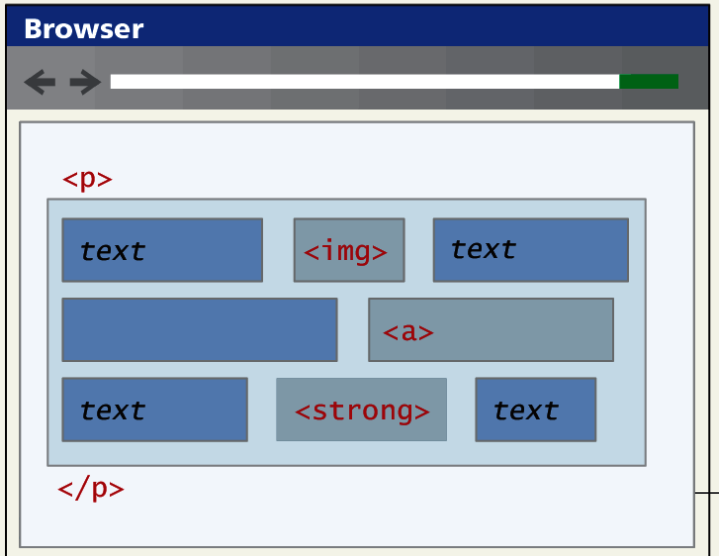
```
<p>
This photo <img src="photo-con.png" alt="" /> of Conservatory Pond in
<a href="http://www.centralpark.com/">Central Park</a> New York City
was taken on October 22, 2015 with a <strong>Canon EOS 30D</strong>
camera.
</p>
```



Inline content is laid out horizontally left to right within its container.

Once a line is filled with content, the next line will receive the remaining content, and so on.

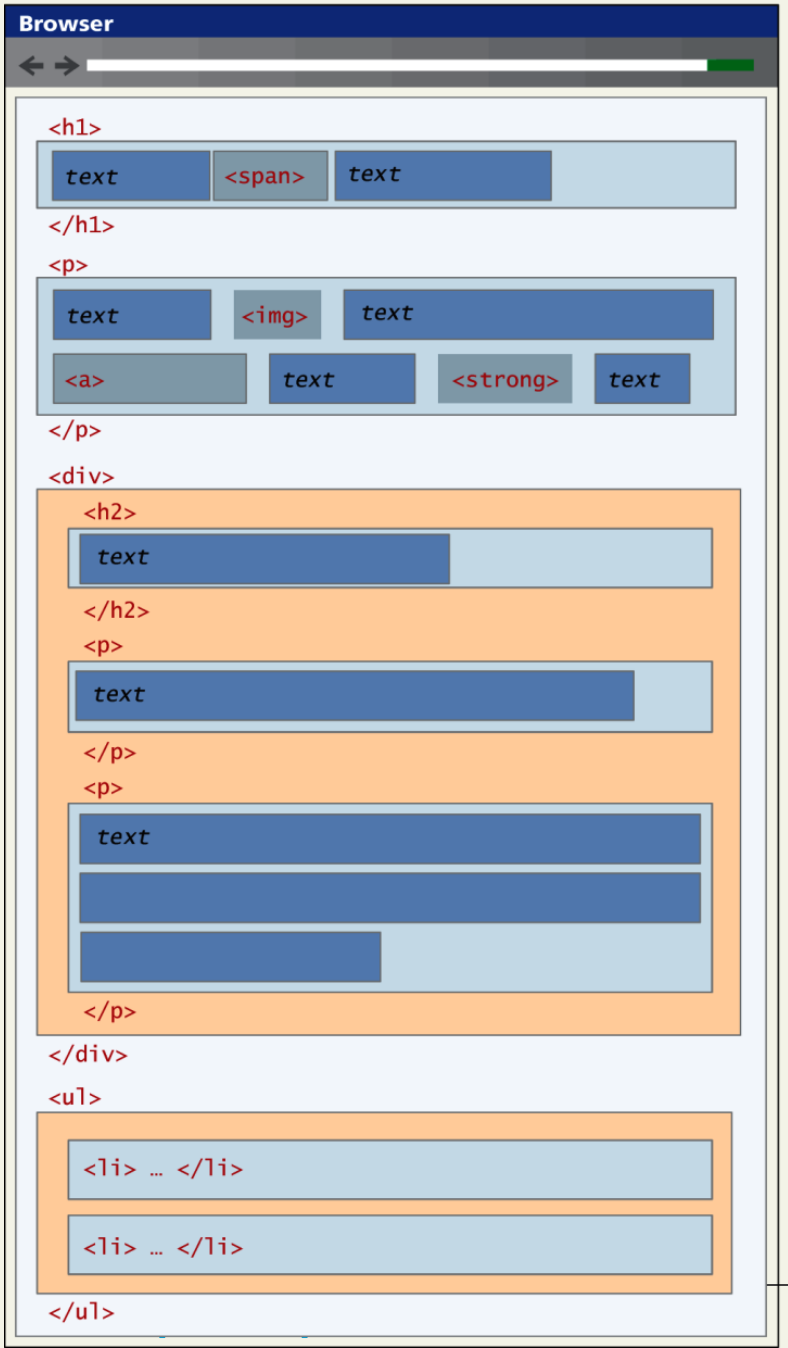
Here the content of this <p> element is displayed on two lines.



If the browser window resizes, then inline content will be "reflowed" based on the new width.

Here the content of this <p> element is now displayed on three lines.

Working Together



A document consists of block-level elements stacked from top to bottom.

Within a block, inline content is horizontally placed left to right.

Some block-level elements can contain other block-level elements (in this example, a `<div>` can contain other blocks).

In such a case, the block-level content inside the parent is stacked from top to bottom within the container (`<div>`).

Take control

It is possible to change whether an element is block-level or inline via the CSS **display** property. Consider the following two CSS rules:

```
span { display: block; }

li { display: inline; }
```

These two rules will make all elements behave like block-level elements and all <li> elements like inline (that is, each list item will be displayed on the same line).

Section 2 of 7

POSITIONING ELEMENTS

Positioning Elements

Move it a little left…

It is possible to move an item from its regular position in the normal flow, and

- move an item outside of the browser viewport so it is not visible
- position it so it is always visible in a fixed position while the rest of the content scrolls.

The **position** property is used to specify the type of positioning

Positioning

Value	Description
absolute	The element is removed from normal flow and positioned in relation to its nearest positioned ancestor.
fixed	The element is fixed in a specific position in the window even when the document is scrolled.
relative	The element is moved relative to where it would be in the normal flow.
static	The element is positioned according to the normal flow. This is the default.

Relative Positioning

Relative to what

In **relative positioning** an element is displaced out of its normal flow position and moved relative to where it would have been placed

The other content around the relatively positioned element "remembers" the element's old position in the flow; thus the space the element would have occupied is preserved

Relative Positioning

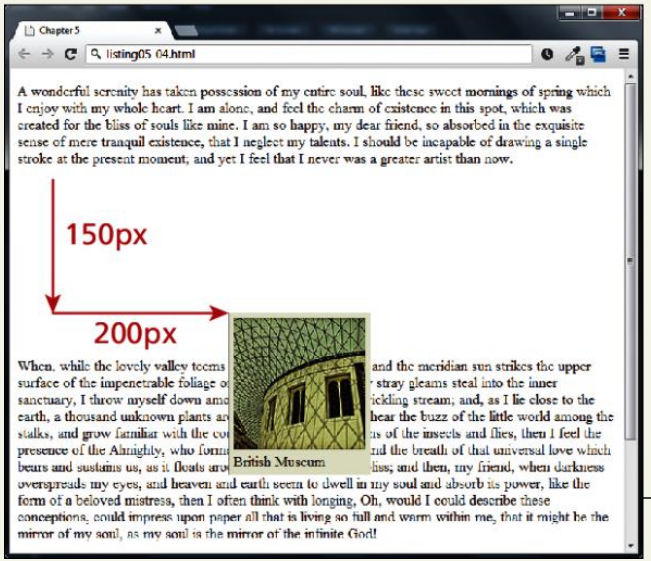
Relatively simple Example




```
<p>A wonderful serenity has taken possession of my …

<figure>
   <img src="images/828.jpg" alt="" />
   <figcaption>British Museum</figcaption>
</figure>

<p>When, while the lovely valley …
```




```
figure {
   border: 1pt solid #A8A8A8;
   background-color: #EDEDDD;
   padding: 5px;
   width: 150px;
   position: relative;
   top: 150px;
   left: 200px;
}
```

Absolute Positioning

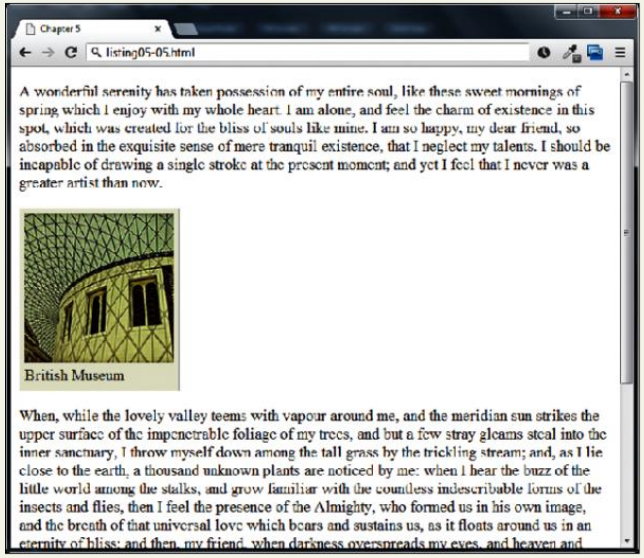
Absolutely!

When an element is positioned absolutely, it is removed completely from normal flow.

Unlike with relative positioning, space is not left for the moved element, as it is no longer in the normal flow.

Absolute Positioning

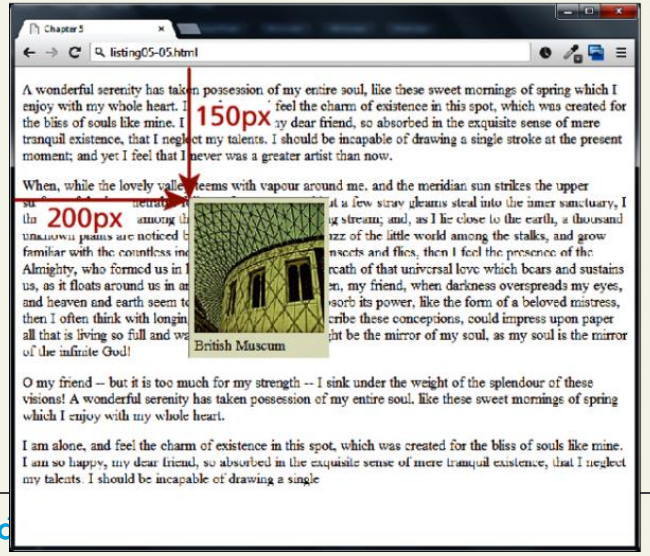
Absolutely great example




```
<p>A wonderful serenity has taken possession of my …

<figure>
   <img src="images/828.jpg" alt="" />
   <figcaption>British Museum</figcaption>
</figure>

<p>When, while the lovely valley …
```




```
figure {
   margin: 0;
   border: 1pt solid #A8A8A8;
   background-color: #EDEDDD;
   padding: 5px;
   width: 150px;
   position: absolute;
   top: 150px;
   left: 200px;
}
```

Absolute Positioning

Absolutely relative?

A moved element via absolute position is actually positioned relative to its nearest **positioned** ancestor container

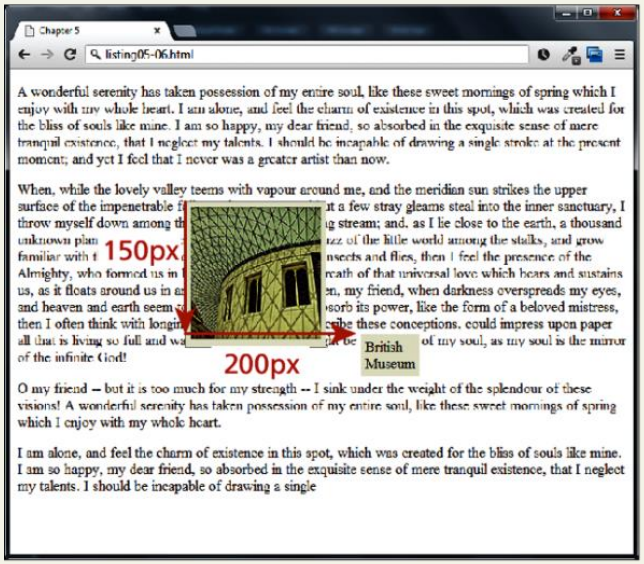
Absolute Positioning




```
<p>A wonderful serenity has taken possession of my …

<figure>
   <img src="images/828.jpg" alt="" />
   <figcaption>British Museum</figcaption>
</figure>

<p>When, while the lovely valley …
```




```
figure {
   margin: 0;
   border: 1pt solid #A8A8A8;
   background-color: #EDEDDD;
   padding: 5px;
   width: 150px;
   position: absolute;
   top: 150px;
   left: 200px;
}

figcaption {
   background-color: #EDEDDD;
   padding: 5px;
   position: absolute;
   top: 150px;
   left: 200px;
}
```

Z-Index

So, like the last topic?

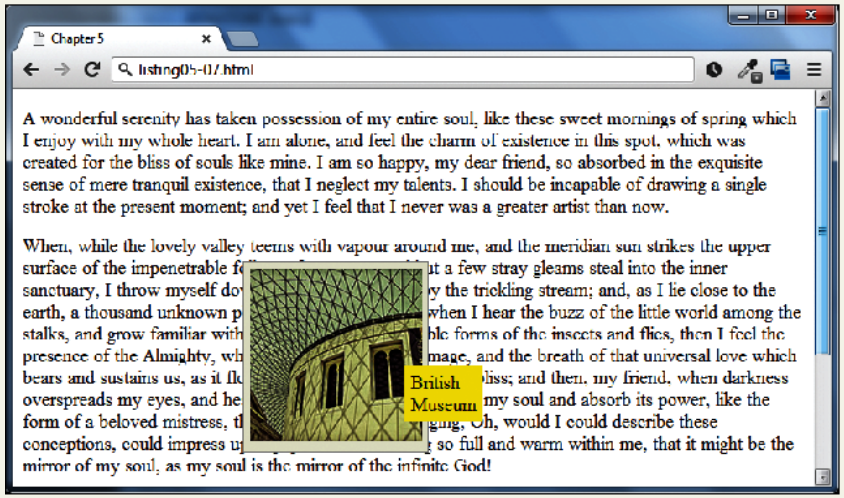
When overlapping elements items closest to the viewer

(and thus on the top) have a larger **z-index**

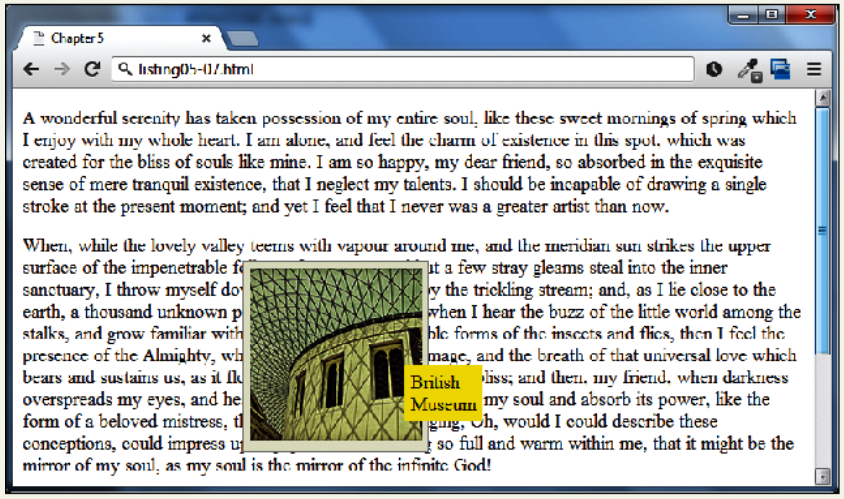
- only positioned elements will make use of their z-index
- simply setting the z-index value of elements will not necessarily move them on top or behind other items.

Z-index

Example




```
figure {
    position: absolute;
    top: 150px;
    left: 200px;
}
figcaption {
    position: absolute;
    top: 90px;
    left: 140px;
}
```

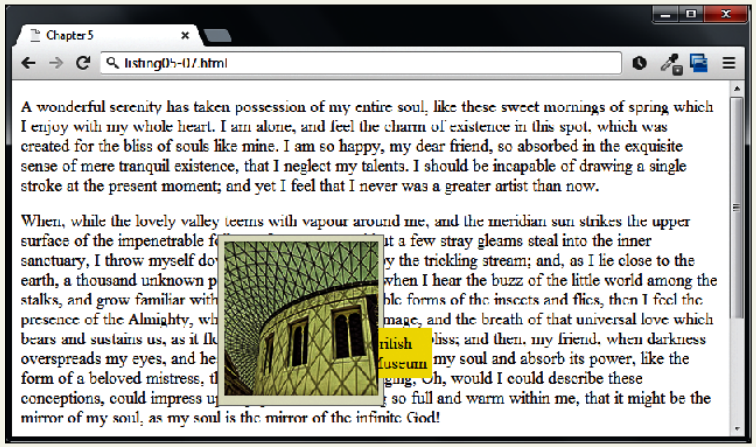



```
figure {
    …
    z-index: 5;
}
figcaption {
    …
    z-index: 1;
}
```

Note that this did **not** move the `<figure>` on top of the `<figcaption>` as one might expect. This is due to the nesting of the caption within the figure.

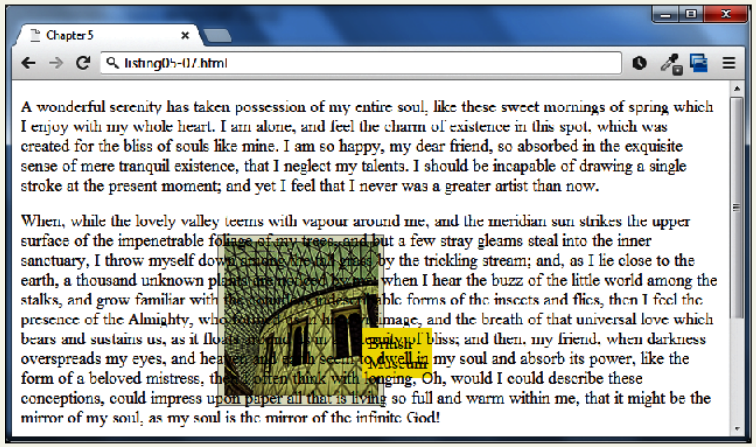
Z-index

Example




```
figure {
    …
    z-index: 1;
}
figcaption {
    …
    z-index: -1;
}
```

Instead the `<figcaption>` `z-index` must be set below 0. The `<figure>` `z-index` could be any value equal to or above 0.




```
figure {
    …
    z-index: -1;
}
figcaption {
    …
    z-index: 1;
}
```

If the `<figure>` `z-index` is given a value less than 0, then any of its positioned descendants change as well. Thus both the `<figure>` and `<figcaption>` move underneath the body text.

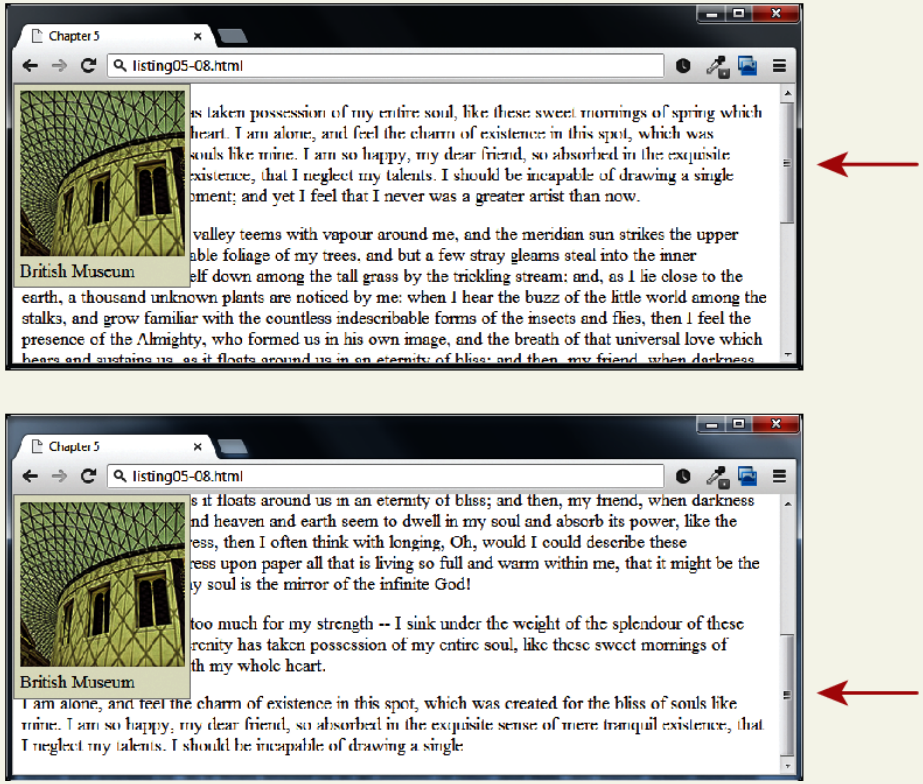
Fixed Position

Place your annoying ads here

Elements with **fixed positioning** do not move when the user scrolls up or down the page

```
figure {
    ...
    position: fixed;
    top: 0;
    left: 0;
}
```

Notice that figure is fixed in its position regardless of what part of the page is being viewed.



Section 3 of 7

FLOATING ELEMENTS

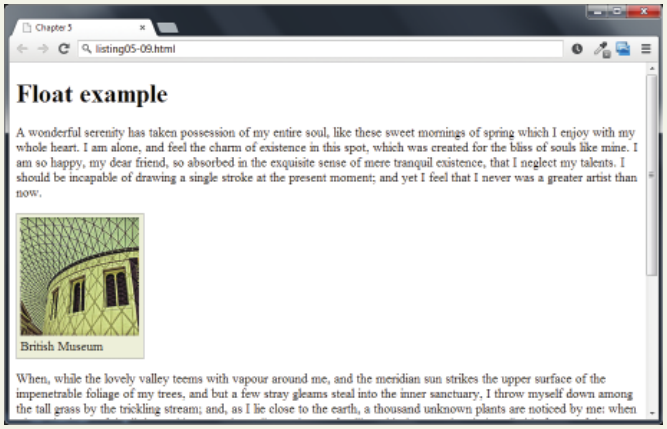
Floating Elements

- It is possible to displace an element out of its position in the normal flow via the CSS **float** property
- An element can be floated to the **left** or floated to the **right**
- When an item is floated, it is moved all the way to the far left or far right of its containing block and the rest of the content is “re-flowed” around the floated element

Floating Elements

Value	Description
left	The left-hand edge of the element cannot be adjacent to another element.
right	The right-hand edge of the element cannot be adjacent to another element.
none	The element can be adjacent to other elements.

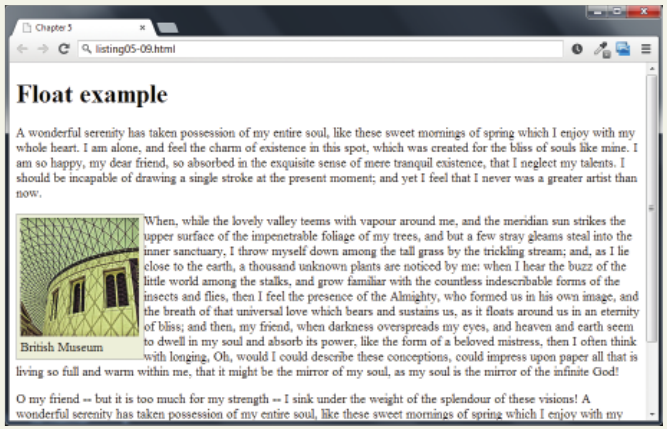
Floating Elements



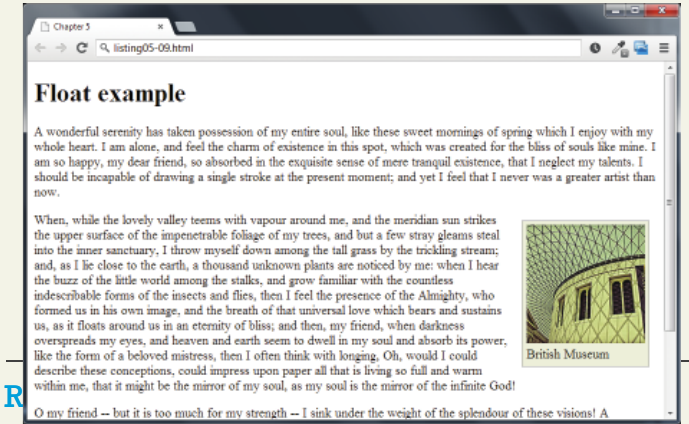

```
<h1>Float example</h1>
<p>A wonderful serenity has taken ...</p>
<figure>
   <img src="images/828.jpg" alt="..." />
   <figcaption>British Museum</figcaption>
</figure>
<p>When, while the lovely valley …</p>
```

```
figure {
   border: 1pt solid #A8A8A8;
   background-color: #EDEDDD;
   margin: 0;
   padding: 5px;
   width: 150px;
}
```

Notice that a floated block-level element **should** have a width specified.




```
figure {
   …
   width: 150px;
   float: left;
}
```




```
figure {
   …
   width: 150px;
   float: right;
   margin: 10px;
}
```

Floating Elements

Details

Notice that a floated block-level element must have a width specified, if you do not, then the width will be set to auto, which will mean it implicitly fills the entire width of the containing block, and there thus will be no room available to flow content around the floated item.

It should be reiterated that a floated item moves to the left or right of its container (also called its **containing block**).

Floating within a container

```
<article>
  <h1>Float example</h1>
  <p>A wonderful serenity has taken possession of … </p>

  <figure>
    <img src="images/828.jpg" alt="" />
    <figcaption>British Museum</figcaption>
  </figure>

  <p>When, while the lovely valley teems with ...</p>

  <p>O my friend -- but it is too much for my ...</p>
</article>
```

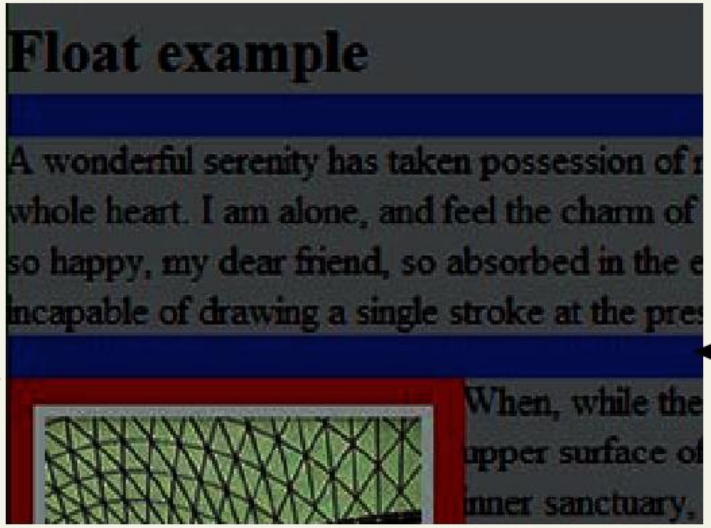



```
article {
   background-color: #898989;
   margin: 5px 50px;
   padding: 5px 20px;
}
p { margin: 16px 0; }
figure {
   border: 1pt solid #262626;
   background-color: #c1c1c1;
   padding: 5px;
   width: 150px;
   float: left;
   margin: 10px;
}
```



Floating within a container

Zoom



<figure> margin-top: 10px;

But these margins did **not** collapse.

<p> margin-bottom: 16px;

Notice these margins collapse (normal behavior).

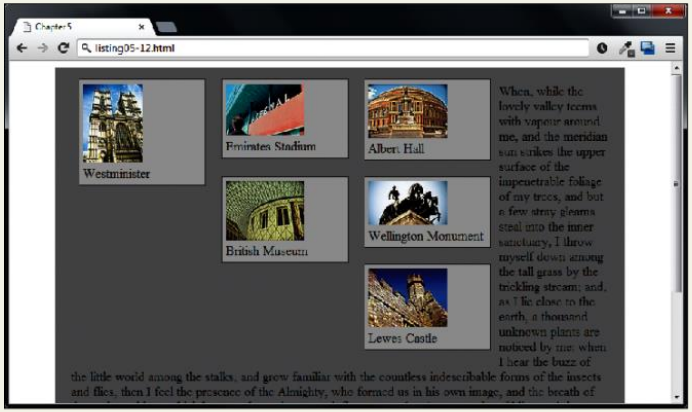
<p> margin-top: 16px;

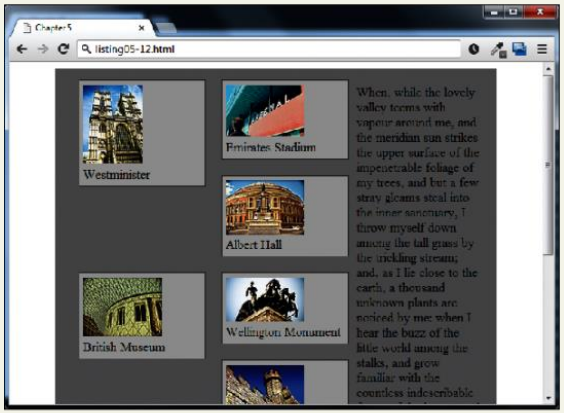
Floating Multiple Items side by side

When you float multiple items that are in proximity, each floated item in the container will be nestled up beside the previously floated item

All other content in the containing block (including other floated elements) will flow around all the floated elements

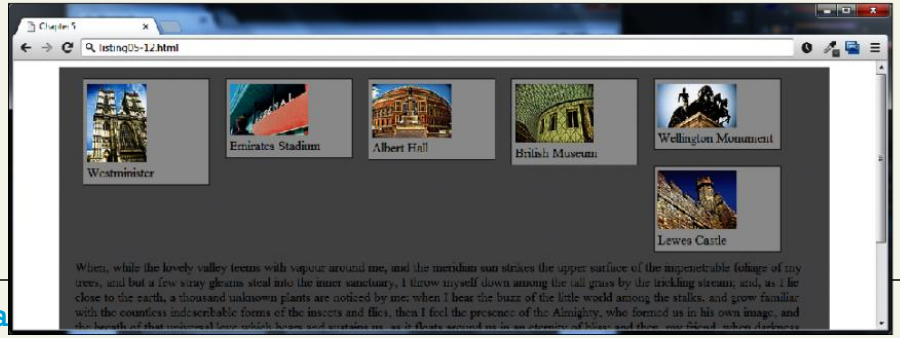
Floating side by side






```
<article>
  <figure>
     <img src="images/tiny/275.jpg" alt="" />
     <figcaption>Westminister</figcaption>
  </figure>
  <figure>
     <img src="images/tiny/700.jpg" alt="" />
     <figcaption>Emirates Stadium</figcaption>
  </figure>
  <figure>
     <img src="images/tiny/537.jpg" alt="" />
     <figcaption>Albert Hall</figcaption>
  </figure>
  <figure>
     <img src="images/tiny/828.jpg" alt="" />
     <figcaption>British Museum</figcaption>
  </figure>
  <figure>
     <img src="images/tiny/464.jpg" alt="" />
     <figcaption>Wellington Monument</figcaption>
  </figure>
  <figure>
     <img src="images/tiny/224.jpg" alt="" />
     <figcaption>Lewes Castle</figcaption>
  </figure>
  <p>When, while the lovely valley teems ..
</article>
```

```
figure {
   ...
   width: 150px;
   float: left;
}
```

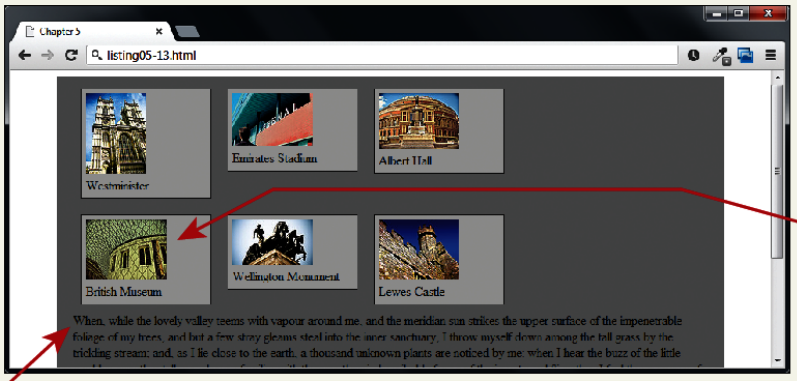


As the window resizes, the content in the containing block (the `<article>` element), will try to fill the space that is available to the right of the floated elements.

The Clear Property

You can stop elements from flowing around a floated element by using the **clear** property

```
.first { clear: left; }
```




```
<article>
  <figure>
     <img src="images/tiny/275.jpg" alt="" />
     <figcaption>Westminister</figcaption>
  </figure>
  <figure>
     <img src="images/tiny/700.jpg" alt="" />
     <figcaption>Emirates Stadium</figcaption>
  </figure>
  <figure>
     <img src="images/tiny/537.jpg" alt="" />
     <figcaption>Albert Hall</figcaption>
  </figure>
  <figure class="first">
     <img src="images/tiny/828.jpg" alt="" />
     <figcaption>British Museum</figcaption>
  </figure>
  <figure>
     <img src="images/tiny/464.jpg" alt="" />
     <figcaption>Wellington Monument</figcaption>
  </figure>
  <figure>

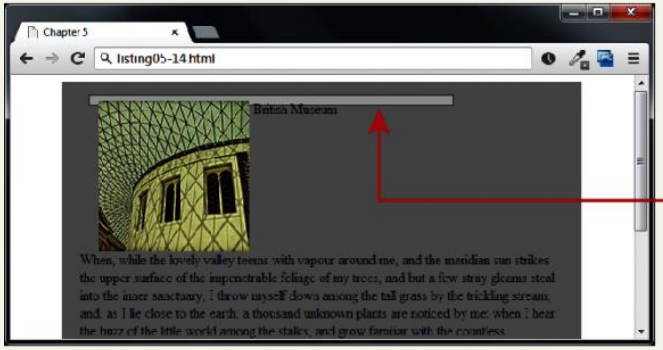
     <img src="images/tiny/224.jpg" alt="" />
     <figcaption>Lewes Castle</figcaption>
  </figure>
  <p class="first">When, while the lovely valley ..
</article>
```

Containing Floats

Problem

Another problem that can occur with floats is when an element is floated within a containing block that contains *only* floated content. In such a case, the containing block essentially disappears

```
<article>
 <figure>
   <img src="images/828.jpg" alt="" />
   <figcaption>British Museum</figcaption>
 </figure>
 <p class="first">When, while the lovely valley …
</article>
```



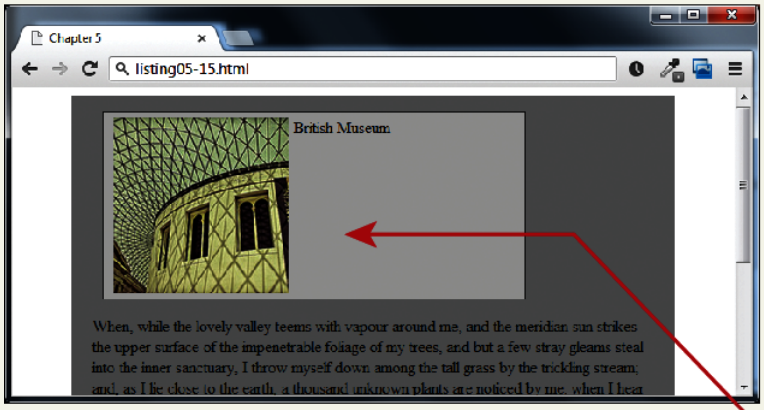
Notice that the <figure> element's content area has shrunk down to zero (it now just has padding space and borders).

```
figure img {
   width: 170px;
   float: left;
   margin: 0 5px;
}
figure figcaption {
   width: 100px;
   float: left;
}
figure {
   border: 1pt solid #262626;
   background-color: #c1c1c1;
   padding: 5px;
   width: 400px;
   margin: 10px;
}
.first { clear: left; }
```

Containing Floats

Solution

One solution would be to float the container as well, but depending on the layout this might not be possible. A better solution would be to use the **overflow** property



Setting the `overflow` property to `auto` solves the problem.

```
figure img {
   width: 170px;
   float: left;
   margin: 0 5px;
}
figure figcaption {
   width: 100px;
   float: left;
}
figure {
   border: 1pt solid #262626;
   background-color: #c1c1c1;
   padding: 5px;
   width: 400px;
   margin: 10px;
   overflow: auto;
}
```

Overlaying and Hiding Elements

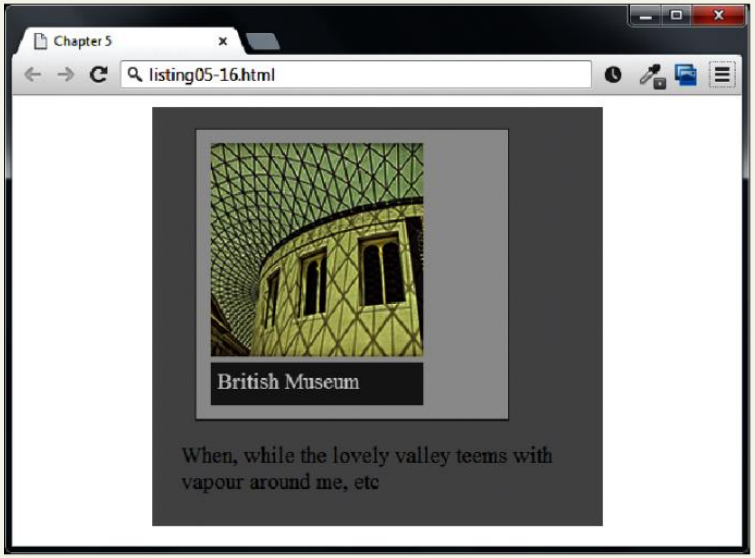
One of the more common design tasks with CSS is to place two elements on top of each other, or to selectively hide and display elements. Positioning is important to both of these tasks.

Positioning is often used for smaller design changes, such as moving items relative to other elements within a container

In such a case, relative positioning is used to create the **positioning context** for a subsequent absolute positioning move

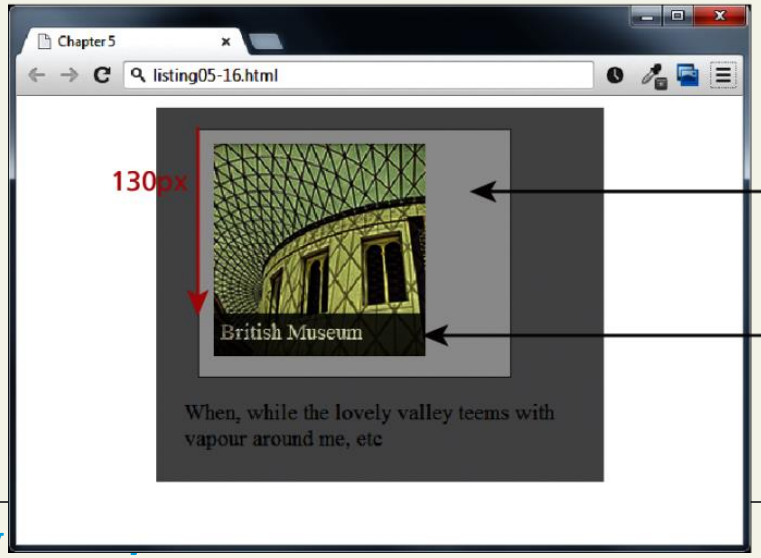
Positioning Context

A Helpful thing to learn




```
figure {
   border: 1pt solid #262626;
   background-color: #c1c1c1;
   padding: 10px;
   width: 200px;
   margin: 10px;
}

figcaption {
   background-color: black;
   color: white;
   opacity: 0.6;
   width: 140px;
   height: 20px;
   padding: 5px;
}
```




```
figure {
   ...
   position: relative;
}
figcaption {
   ...
   position: absolute;
   top: 130px;
   left: 10px;
}
```

This creates the positioning context.

This does the actual move.

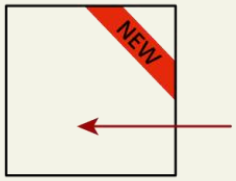
Positioning Context

A realistic example

```
<figure>
   <img src="images/828.jpg" alt="" />
   <figcaption>British Museum</figcaption>
   <img src="images/new-banner.png" alt="" class="overlayed"/>
</figure>
```

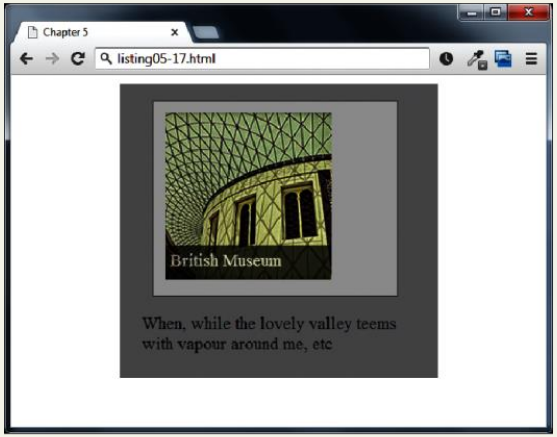



```
.overlayed {
   position: absolute;
   top: 10px;
   left: 10px;
}
```



Transparent area

new-banner.png




```
.overlayed {
   position: absolute;
   top: 10px;
   left: 10px;
   display: none;
}
```

This hides the overlayed image.

```
.hide {
   display: none;
}
```

This is the preferred way to hide: by adding this class to another element. This makes it clear in the markup that an element is not visible.

```
<img ... class="overlayed hide"/>
```

Hiding elements

Two different ways to hide elements in CSS:

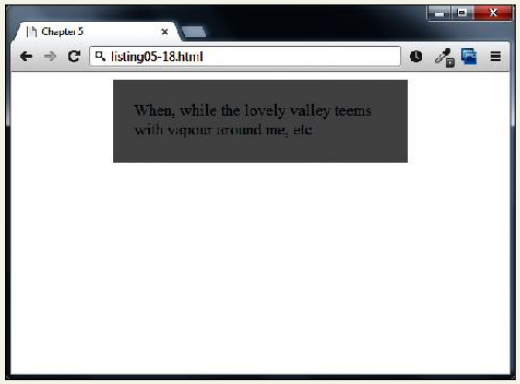
1. using the **display** property
    - The display property takes an item out of the flow: it is as if the element no longer exists
2. using the **visibility** property
    - The visibility property just hides the element, but the space for that element remains.

Hiding elements

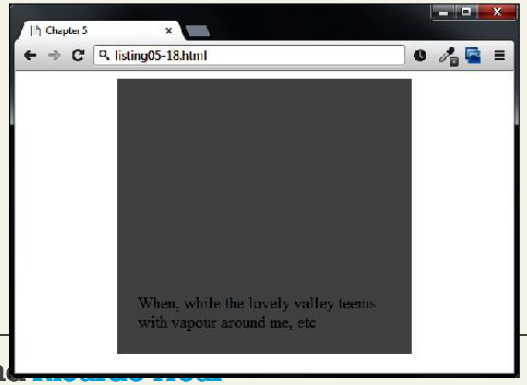
Two ways to hide: display and visibility




```
figure {
    ...
    display: auto;
}
```




```
figure {
    ...
    display: none;
}
```



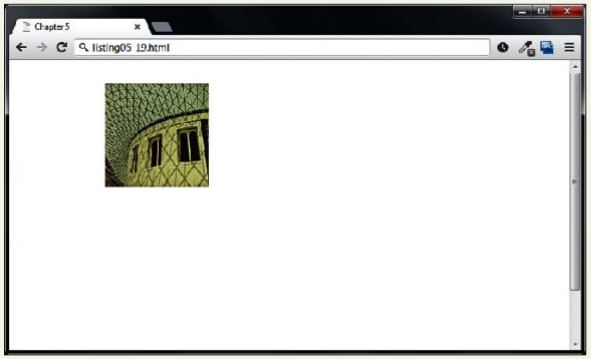

```
figure {
    ...
    visibility: hidden;
}
```

Using :hover to make thumbnails

Next slide shows how the combination of absolute positioning, the **:hover** pseudo-class, and the visibility property can be used to display a larger version of an image

Using :hover to make thumbnails

```
<figure class="thumbnail">
  <img src="images/828.jpg" alt="" />
  <figcaption class="popup">
     <img src="images/828-bigger.jpg" alt="" />
     <p>The library in the British Museum in London</p>
  </figcaption>
</figure>
```

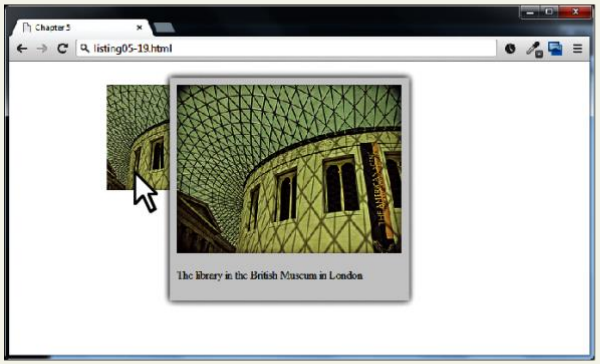


When the page is displayed, the larger version of the image, which is within the `<figcaption>` element, is `hidden`.

```
figcaption.popup {
   padding: 10px;
   background: #e1e1e1;
   position: absolute;

   /* add a drop shadow to the frame */
   -webkit-box-shadow: 0 0 15px #A9A9A9);
   -moz-box-shadow: 0 0 15px #A9A9A9;
   box-shadow: 0 0 15px #A9A9A9;

   /* hide it until there is a hover */
   visibility: hidden;
}
```



When the user moves/hovers the mouse over the thumbnail image, the `visibility` property of the `<figcaption>` element is set to `visible`.

```
figure.thumbnail:hover figcaption.popup {
   position: absolute;
   top: 0;
   left: 100px;

   /* display image upon hover */
   visibility: visible;

}
```

Section 4 of 7

CONSTRUCTING MULTICOLUMN LAYOUTS

Constructing Multicolumn Layouts

There is unfortunately no simple and easy way to create robust multicolumn page layouts. There are tradeoffs with each approach:

- Using Floats to Create Columns
- Using Positioning to Create Columns

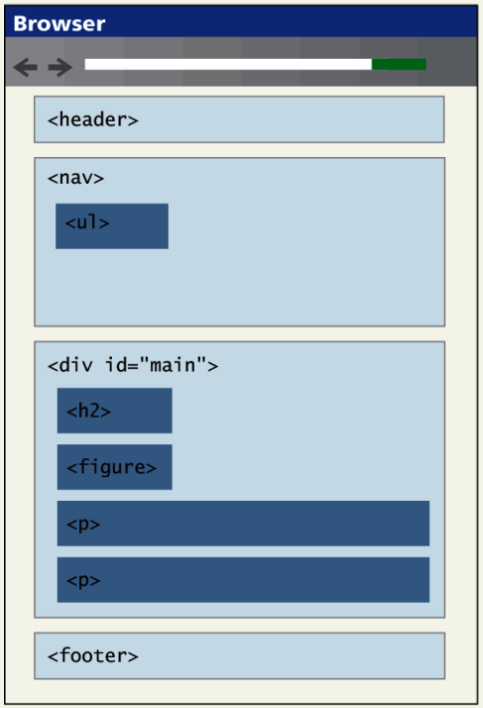
Using Floats to create Columns

The most common way to create columns of content is using floats.

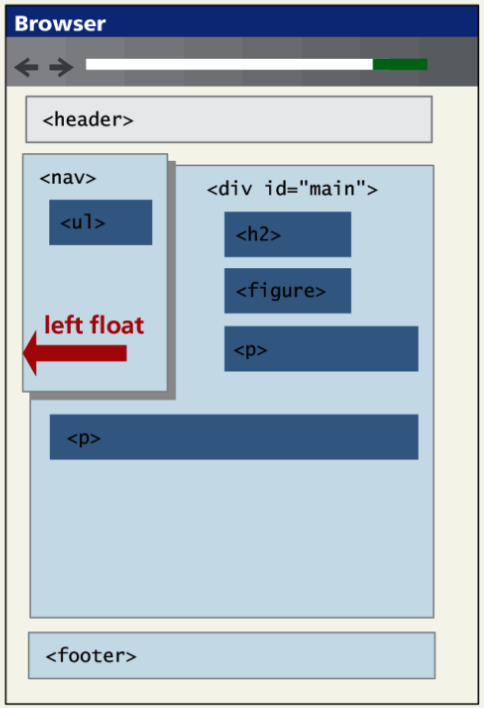
- The first step is to float the content container that will be on the left-hand side. Remember that the floated container needs to have a width specified.

Using Floats to create Columns

1 HTML source order (normal flow)



2 Two-column layout (left float)




```
nav {
  …
  width: 12em;
  float: left;
}
```

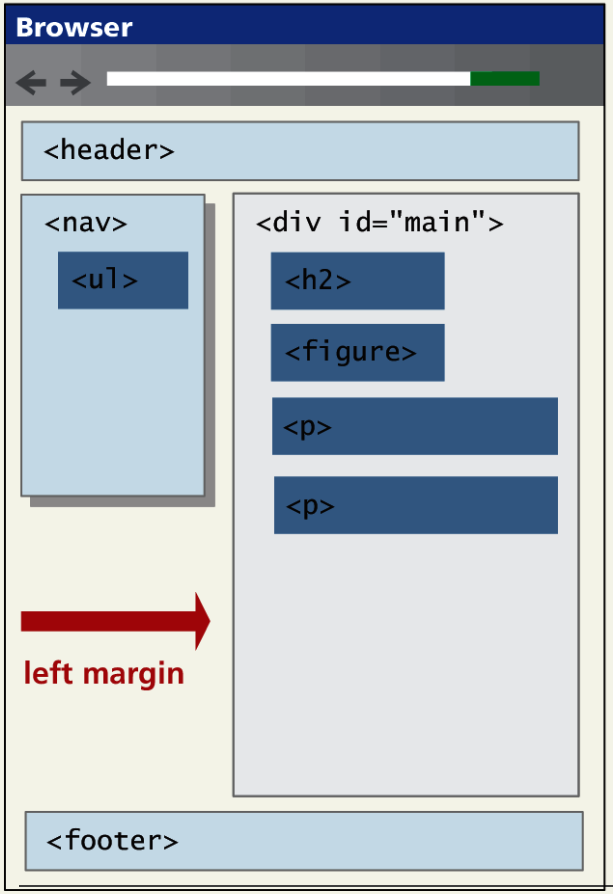


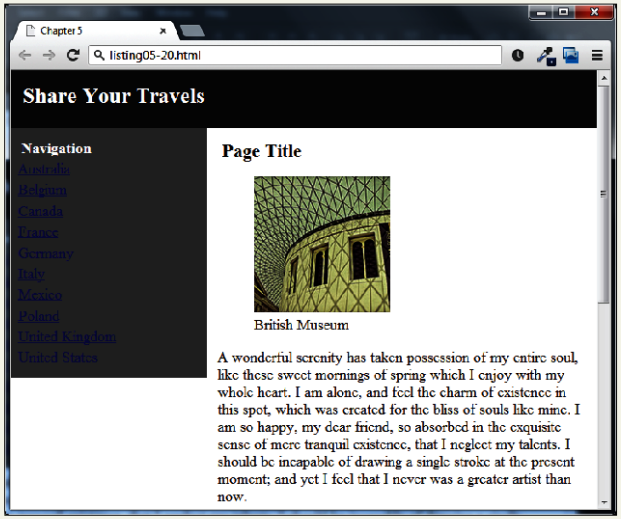


Using Floats to create Columns

The other key step is changing the left-margin so that it no longer flows back under the floated content.

3 Set the left margin of non-floated content

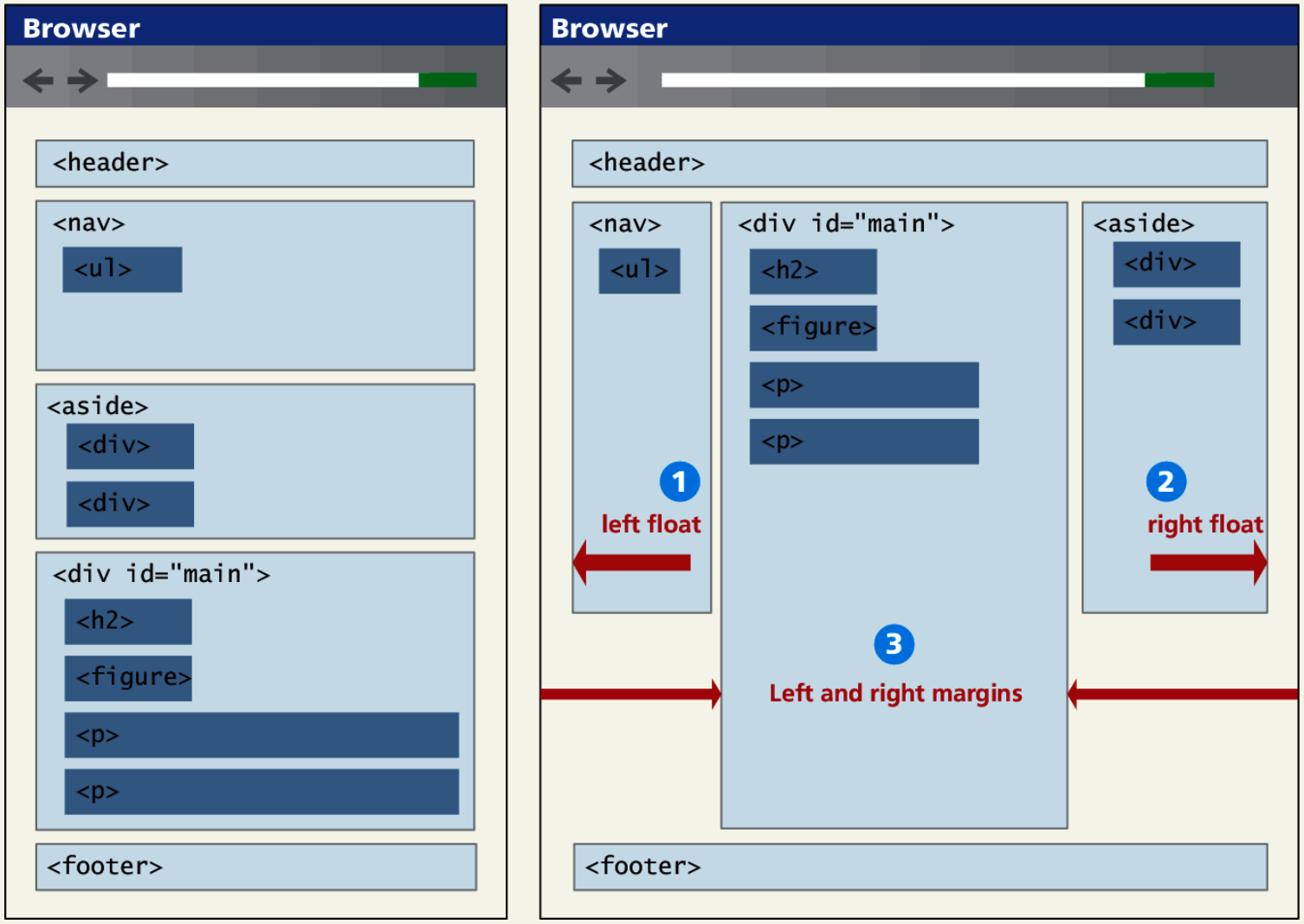




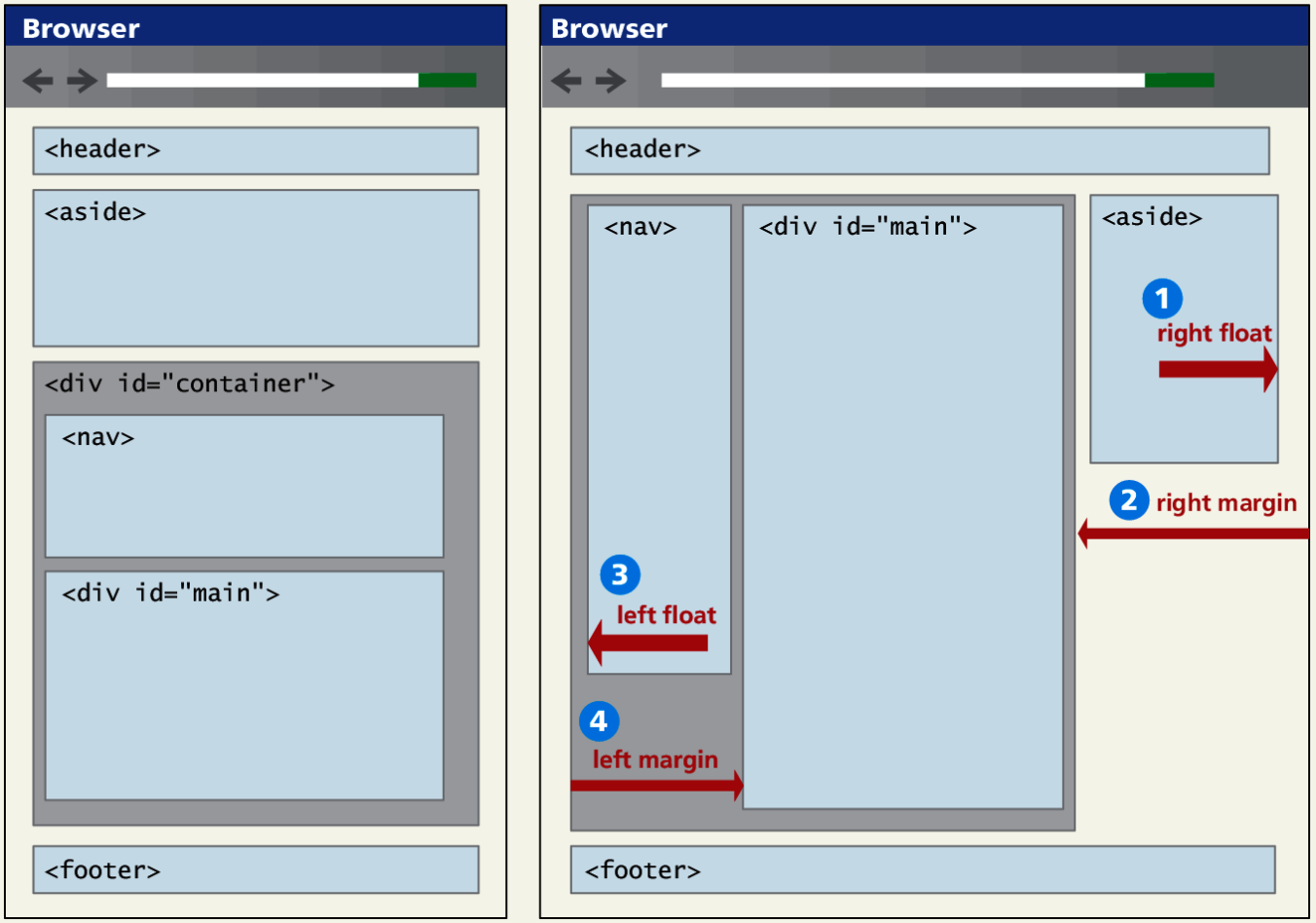

```
div#main {
  …
  margin-left: 220px;
}
```

Using Floats for Columns

3 Column Example

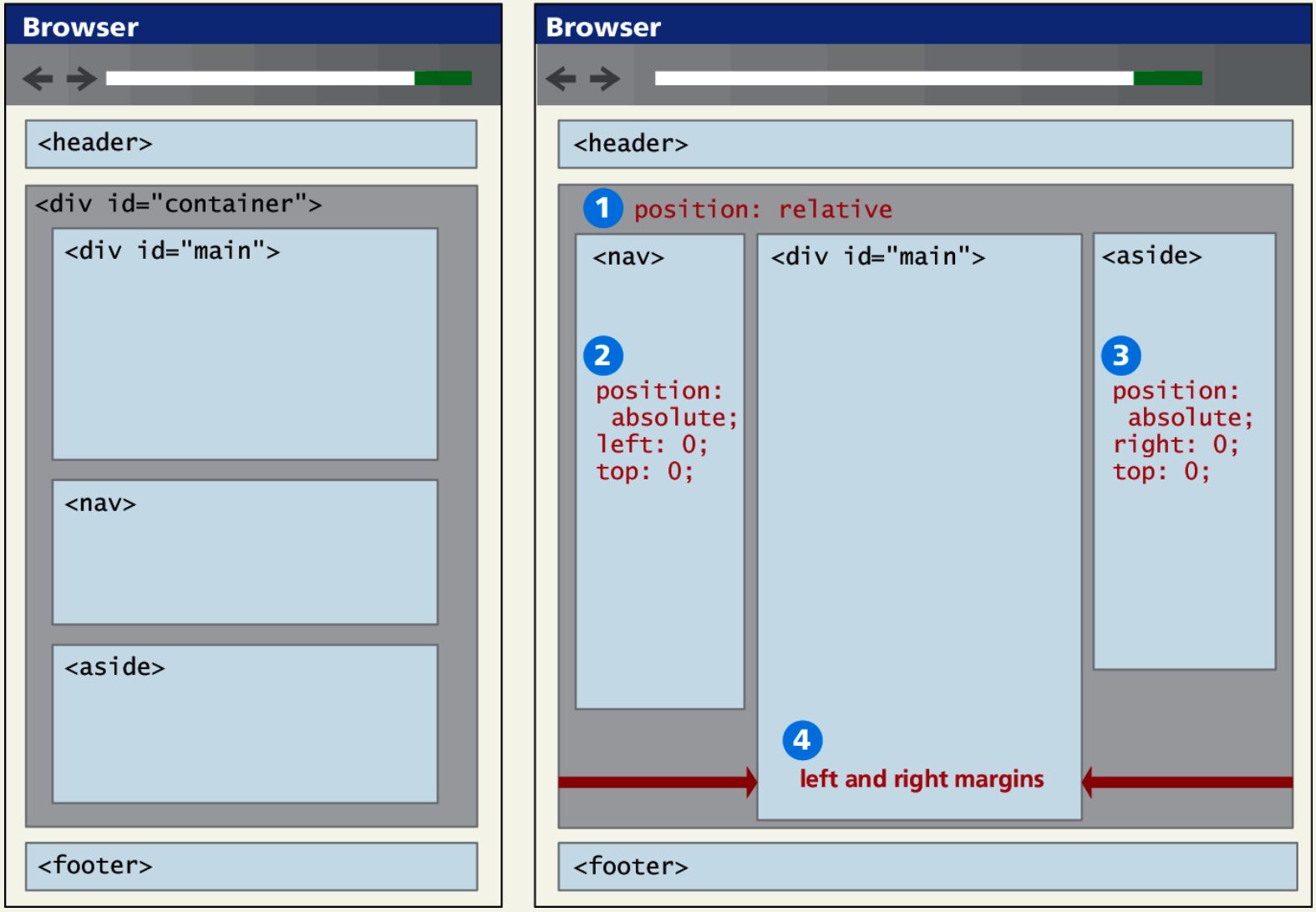


Another example



Positioning discussion

Positives



Positioning discussion

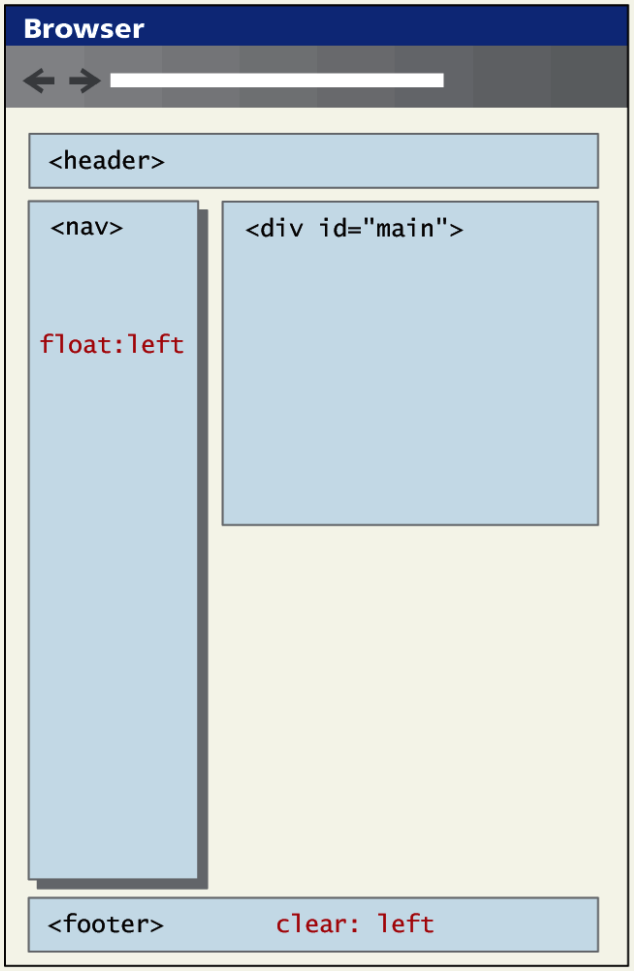
Challenges

What would happen if one of the sidebars had a lot of content and was thus quite long?

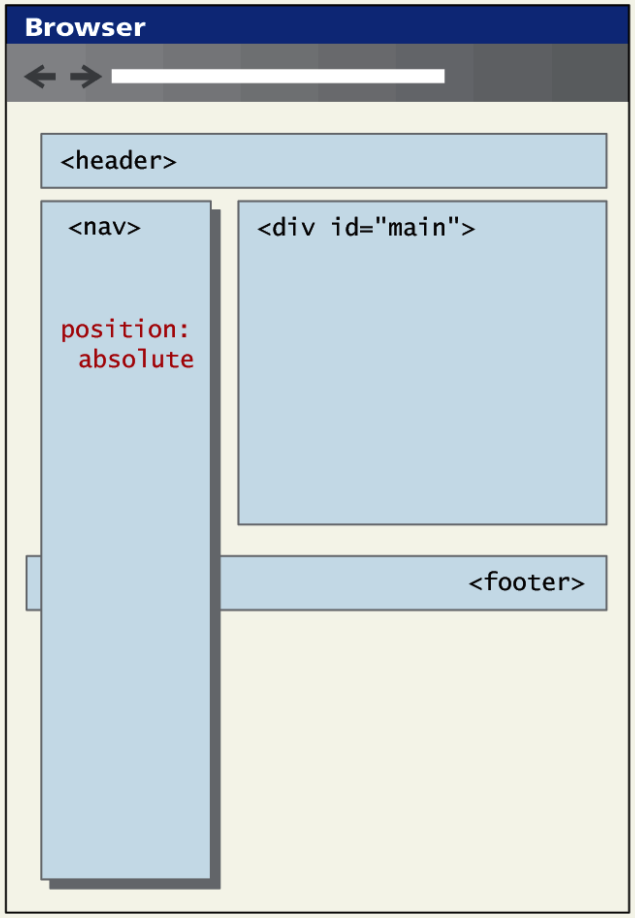
- In the floated layout, this would not be a problem at all, because when an item is floated, blank space is left behind.
- But when an item is positioned, it is removed entirely from normal flow, so subsequent items will have no “knowledge” of the positioned item.

Positioning Discussion

Problem illustration



Elements that are floated leave behind space for them in the normal flow. We can also use the `clear` property to ensure later elements are below the floated element.



Absolute positioned elements are taken completely out of normal flow, meaning that the positioned element may overlap subsequent content. The `clear` property will have no effect since it only responds to floated elements.

Constructing Multicolumn Layout

Solution to footer problem



Constructing Multicolumn Layout

Using Flexbox to Create Columns




```
<div class="media">
   <img class="media-image"
      src="calgary.jpg" alt="test" >
   <div class="media-body">
      <h2>Fall in Calgary</h2>
      <p>Nunc nec fermentum dolor...</p>
      <p>Mauris porta arcu id...</p>
      <p>Phasellus vel felis purus...</p>
   </div>
</div>
```

```
.media-image {
   float: left;
   margin-right: 10px;
}
.media-body {
   margin-left: 160px;
}
```

Prior to flexbox, one would create such a layout within a container using floats plus margins. The problem with this approach is that margins needed to be in pixels and had to exactly match image size. If image size changed (or you wanted same kind of style elsewhere), you had to modify the style.

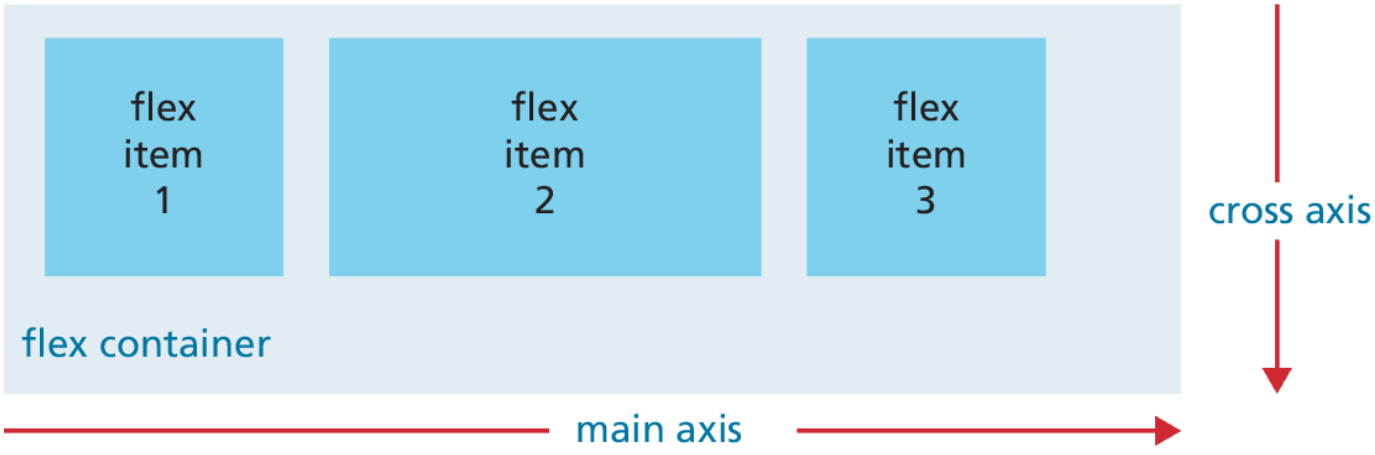



```
.media {
   display: flex;
   align-items: flex-start;
}
.media-image {
   margin-right: 1em;
}
```

Using flexbox, we now have a much more generalized **(and thus reusable)** style.

Flex containers and items

There are two places in which you will be assigning flexbox properties: the **flex container** and the **flex items** within the container.




```
<div class="container">
  <div>flex item 1</div>
  <span>flex item 2</span>
  <div>flex item 3</div>
</div>
```

Both block-level and inline elements will become flex items.

Pearson

Copyright © 2021, 2018, 2015 Pearson Education, Inc. All Rights Reserved

The flexbox container properties

`display: flex` Necessary to use flexbox

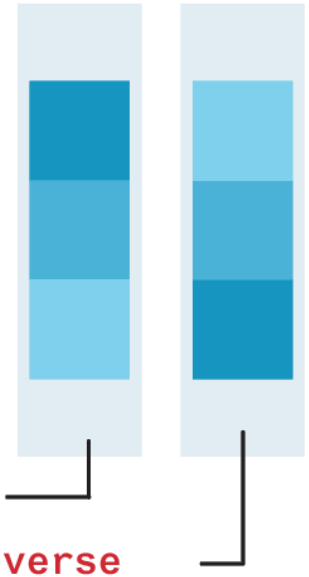
`flex-direction: row`



Specifies the direction of the main axis.
The default is `row`.

`flex-direction: row-reverse`

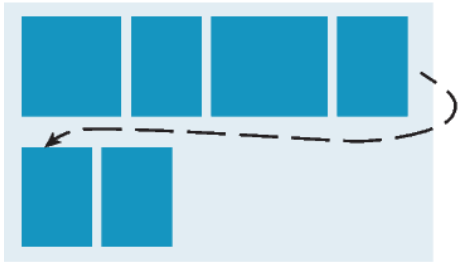




`flex-direction: column`

`flex-direction: column-reverse`

`flex-wrap: wrap`



`flex-wrap: nowrap`



Indicates whether new lines should be created if flex container can't contain all the flex items.

Pearson

Copyright © 2021, 2018, 2015 Pearson Education, Inc. All Rights Reserved

The flexbox container properties (ii)

justify-content: flex-start

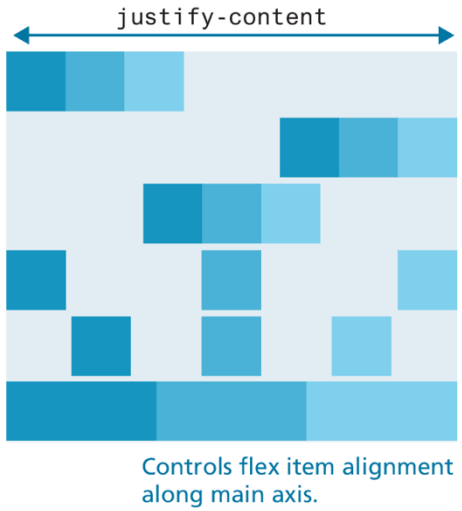
justify-content: flex-end

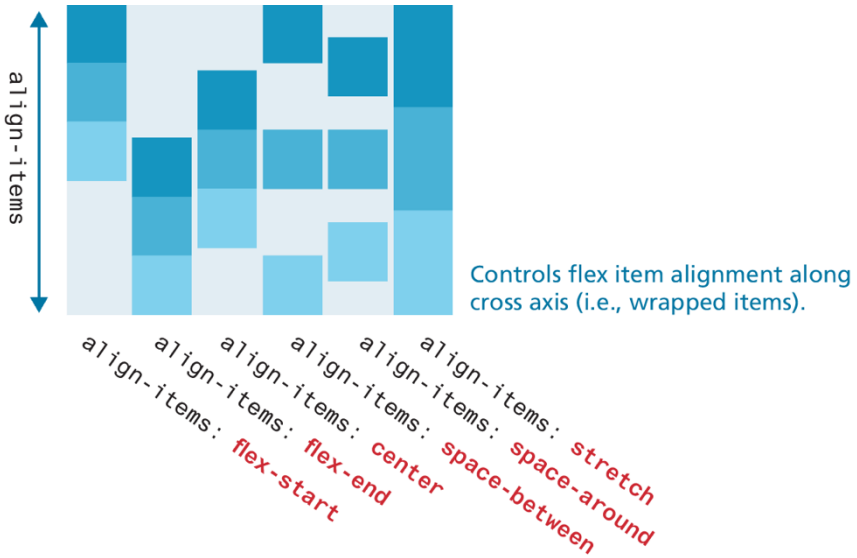
justify-content: center

justify-content: space-between

justify-content: space-around

justify-content: stretch





Pearson

Copyright © 2021, 2018, 2015 Pearson Education, Inc. All Rights Reserved

The flexbox item (child) properties

flex-basis: auto



The flex-basis property determines the initial size of the flex item before the remaining space is distributed.

The default auto value means that the size is determined by the width and height.

```
flex-grow: 1
flex-shrink: 1
flex-basis: auto
```

flex: 1 1 auto

These can be combined into the shorthand property instead

flex-basis: 200px



You can specify a width using px, %, or other measurement units.

flex-grow: 2

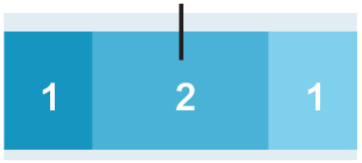


width=*n*

width=*n* × 2

Defines the growth factor of an element relative to the other items.

flex-shrink: 2



Defines how much an item will shrink when not enough space in container.

Pearson

Copyright © 2021, 2018, 2015 Pearson Education, Inc. All Rights Reserved

Flexbox Cards




```
<div class="card">
   <div class="content">
      <img src="w.jpg" />
      <h3>Albert Hall</h3>
   </div>
   <footer>
      <a href="#">View</a>
   </footer>
</div>
```




```
.card {
  ...
  display: flex;
  flex-direction: column;
}

.card .content {
   flex: 1 1 auto;
}
```

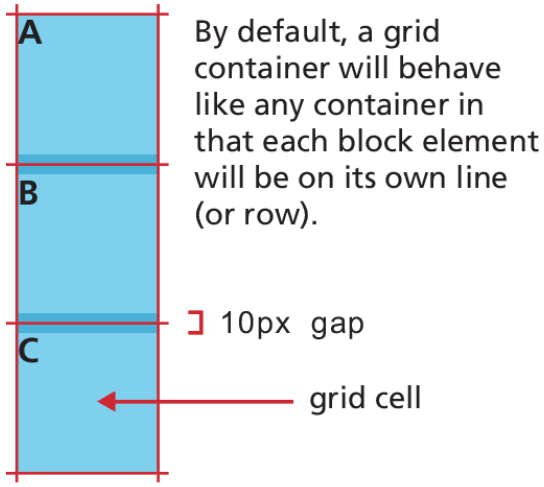
Pearson

Copyright © 2021, 2018, 2015 Pearson Education, Inc. All Rights Reserved

Grid Layout

Grid layout is adjustable, powerful, and, compared to floats, positioning, and even flexbox, is relatively easy to learn and use!

- Each blocklevel child in a parent container whose **display** property is set to **grid** will be automatically placed into a grid cell




```
<div class="container">
  <div>A</div>
  <div>B</div>
  <div>C</div>
  ...
</div>
```

Grid container

The container's block-level children will become the grid items.

```
.container {
  display: grid;
  gap: 10px;
}
```

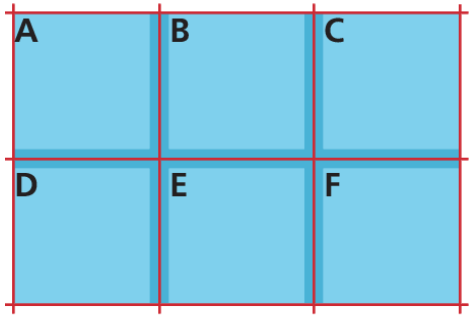
To make each cell more visually distinguisable, we have specified a gap, which adds space around each cell.

Pearson

Copyright © 2021, 2018, 2015 Pearson Education, Inc. All Rights Reserved

Specifying Grid Structure

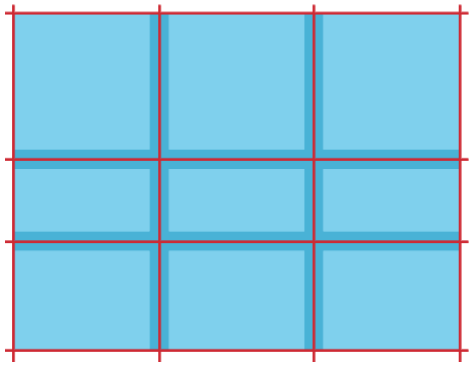
grid-template-columns is used for adding columns by specifying each column's width using the **fr** unit.




```
.container {
  display: grid;
  gap: 10px;
  grid-template-columns: 1fr 1fr 1fr;
}
```

This makes each column equal to an equal fraction of the available space.

You can specify the number of columns per row/line via the `grid-template-column` property.



```
.container {
  ...
  height: 300px;
  grid-template-columns: 1fr 1fr 1fr;
  grid-template-rows: 80px 30px 50px;
}
```

While you often do not need to set a row height, you can make rows taller than their actual content.

Pearson

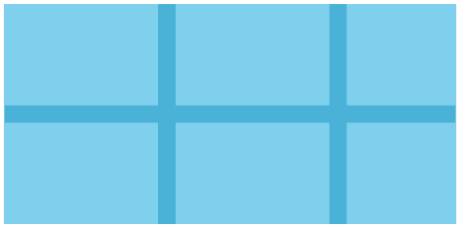
Copyright © 2021, 2018, 2015 Pearson Education, Inc. All Rights Reserved

Specifying column widths

Column widths can be specified

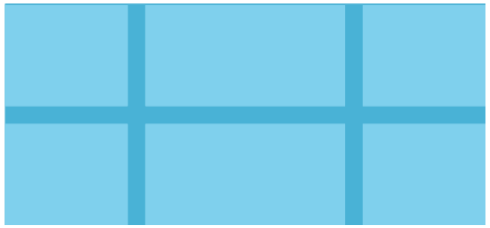
The CSS **repeat()** function provides a way to specify repeating patterns of columns.

```
grid-template-columns: 70px 70px 45px;
```



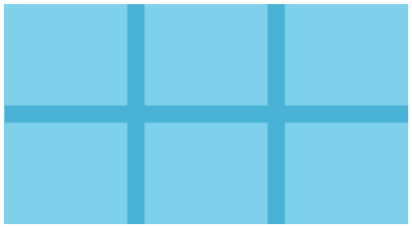
Each column can have its own unique width value, in px, %, em, etc.

```
grid-template-columns: 50px auto 50px;
```



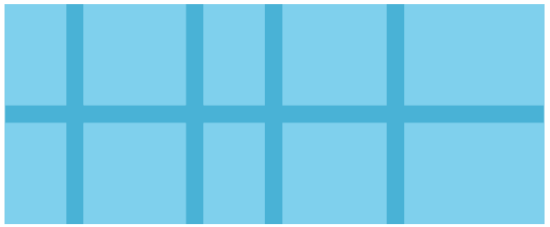
An auto value indicates the width will fill the remaining space.

```
grid-template-columns: repeat(3, 1fr);
```



The repeat function can be used to defining a repeating pattern.

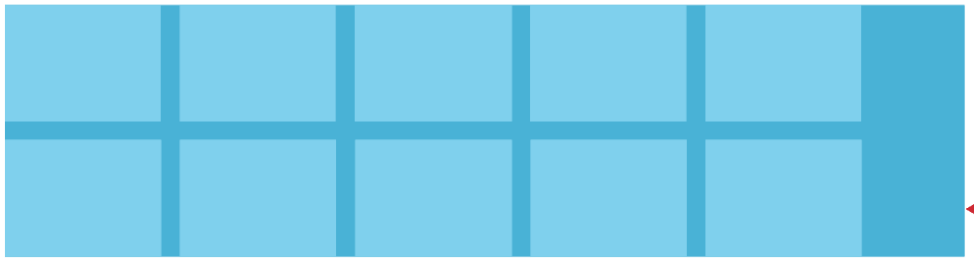
```
grid-template-columns: repeat(2, 30px 50px) 70px;
```



Pearson

Copyright © 2021, 2018, 2015 Pearson Education, Inc. All Rights Reserved

Specifying column widths (ii)

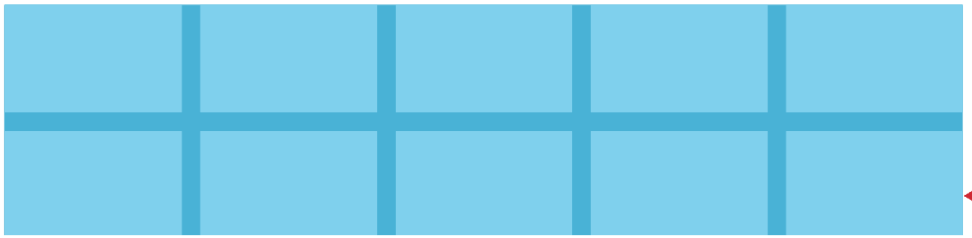


width: 560px;

```
grid-template-columns: repeat(auto-fill, minmax(100px, 1fr));
```

Fills the row with as many columns as can fit into the container space.

The min column size is 100px and the max is whatever size is necessary to allow each column to be equal sized.



width: 560px;

```
grid-template-columns: repeat(auto-fit, minmax(100px, 1fr));
```

Fits the columns into space by expanding the column width into the available container space.

Pearson

Copyright © 2021, 2018, 2015 Pearson Education, Inc. All Rights Reserved

Grid

```
<!-- CSS Grid Approach -->
<div class="container">
  <img src=1.gif />
  <img src=2.gif />
  <img src=3.gif />
  <img src=4.gif />
  <img src=5.gif />
  <img src=6.gif />
</div>

<!-- CSS for grid approach -->
.container {
  display: grid;
  grid-template-columns: repeat(auto-fit,
minmax(100px, 1fr);
}
.container img { display: block; }
```

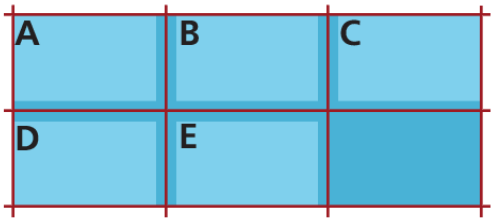
Pearson

Copyright © 2021, 2018, 2015 Pearson Education, Inc. All Rights Reserved

Explicit Grid Placement Example 1

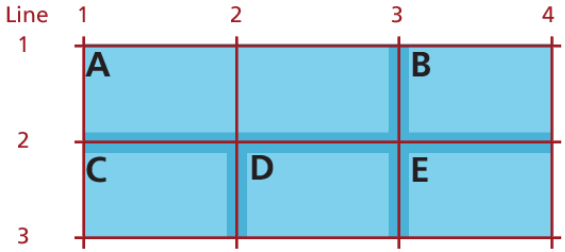
```
<div class="container">
  <div class="a">A</div>
  <div class="b">B</div>
  <div class="c">C</div>
  <div class="d">D</div>
  <div class="e">E</div>
</div>
```

```
.container {
  display: grid;
  gap: 10px;
  grid-template-columns: repeat(3,1fr);
  grid-template-rows: repeat(2,200px);
}
```



With implicit layout, grid items are placed automatically.

Example 1




```
.a {
  grid-column-start: 1;
  grid-column-end: 3;
}
```

The start and end numbers refer to the line number not the column number.

The same effect also possible using either of the following:

```
grid-column: 1 / 3;
grid-column: 1 / span 2;
```

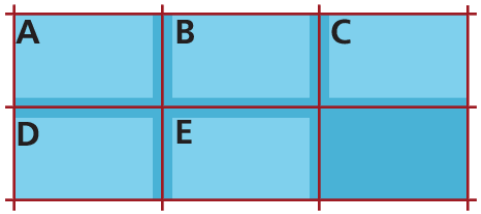
Pearson

Copyright © 2021, 2018, 2015 Pearson Education, Inc. All Rights Reserved

Explicit Grid Placement Example 2

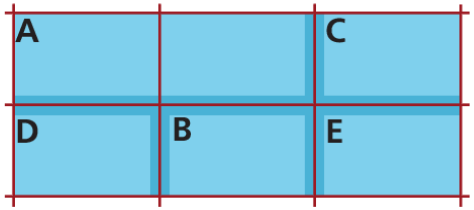
```
<div class="container">
  <div class="a">A</div>
  <div class="b">B</div>
  <div class="c">C</div>
  <div class="d">D</div>
  <div class="e">E</div>
</div>
```

```
.container {
  display: grid;
  gap: 10px;
  grid-template-columns: repeat(3,1fr);
  grid-template-rows: repeat(2,200px);
}
```



With implicit layout, grid items are placed automatically.

2




```
.b {
  grid-row: 2;
  grid-column: 2;
}
```

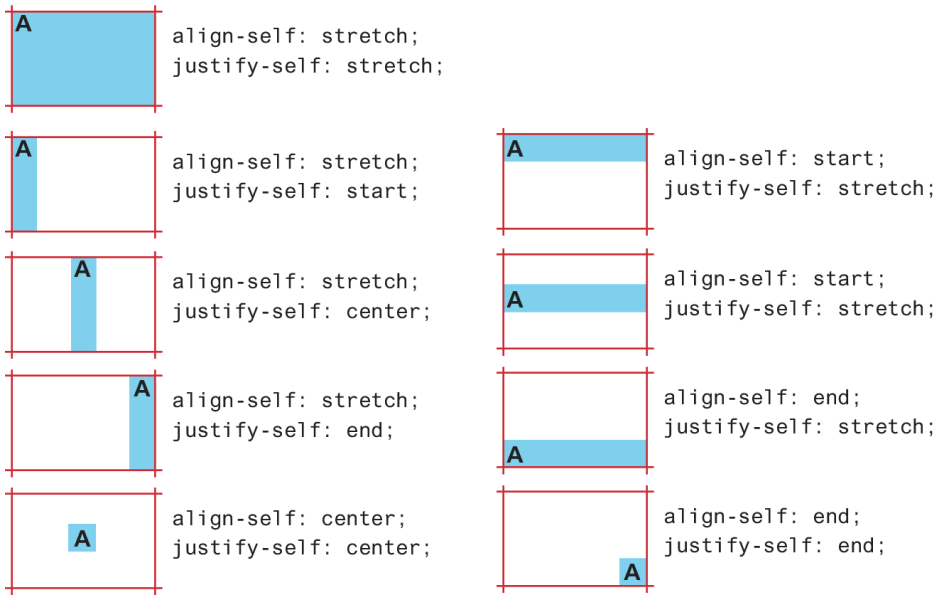
Grid cells can be placed into any row and column.

Pearson

Copyright © 2021, 2018, 2015 Pearson Education, Inc. All Rights Reserved

Cell properties

- **align-self and justify-self** control the cell content's horizontal and vertical alignment within its grid container.



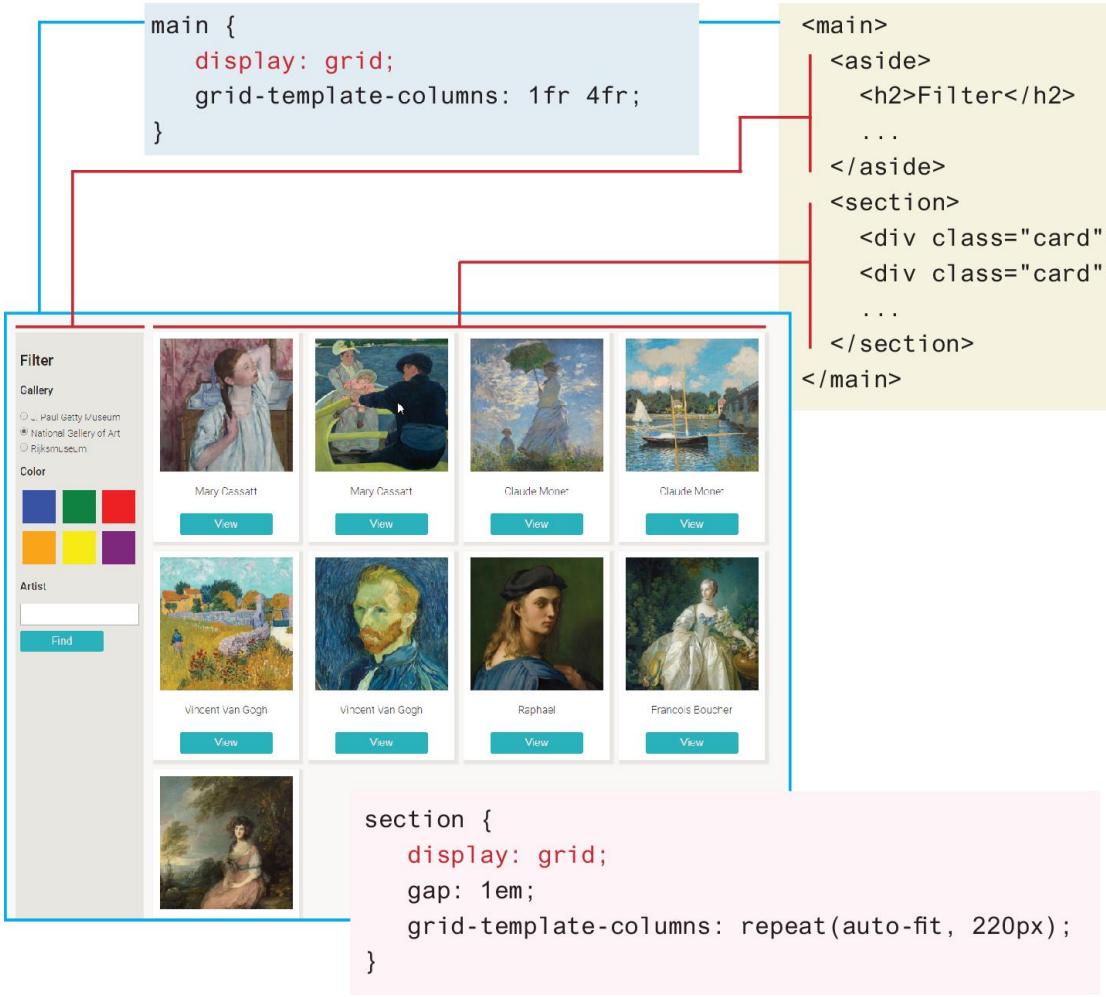
- You can similarily control cell alignment within a grid container using **align-items** and **justify-items**

Pearson

Copyright © 2021, 2018, 2015 Pearson Education, Inc. All Rights Reserved

Nested Grid

- **align-self and justify-self** control the cell content's horizontal and vertical alignment within its grid container.



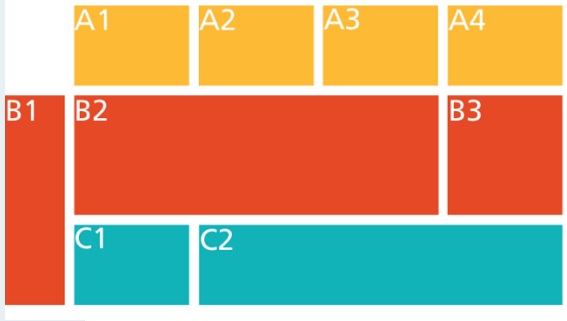
Pearson

Copyright © 2021, 2018, 2015 Pearson Education, Inc. All Rights Reserved

Grid Areas

```
<style>
.container {
  grid-gap: 10px;
  display: grid;
  grid-template-rows: 100px 150px 100px;
  grid-template-columns: 75px 1fr 1fr 1fr 1fr;
  grid-template-areas: ". a1 a2 a3 a4"
                       "b1 b2 b2 b2 b3"
                       "b1 c1 c2 c2 c2";
}
.a1 { grid-area: a1; }
.a2 { grid-area: a2; }
.a3 { grid-area: a3; }
.a4 { grid-area: a4; }
.b1 { grid-area: b1; }
.b2 { grid-area: b2; }
.b3 { grid-area: b3; }
.c1 { grid-area: c1; }
.c2 { grid-area: c2; }
</style>

<section class="container">
  <div class="yellow a1">A1</div>
  <div class="yellow a2">A2</div>
  <div class="yellow a3">A3</div>
  <div class="yellow a4">A4</div>
  <div class="orange b1">B1</div>
  <div class="orange b2">B2</div>
  <div class="orange b3">B3</div>
  <div class="cyan c1">C1</div>
  <div class="cyan c2">C2</div>
</section>
```

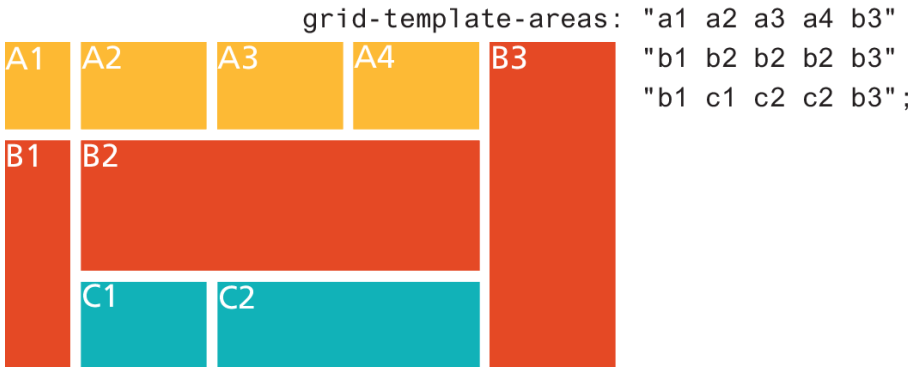


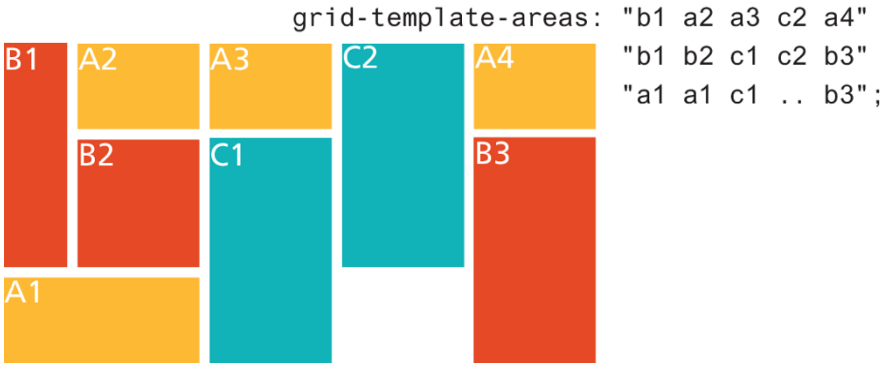
LISTING 7.2 Using grid areas

Pearson

Copyright © 2021, 2018, 2015 Pearson Education, Inc. All Rights Reserved

Grid Areas (ii)





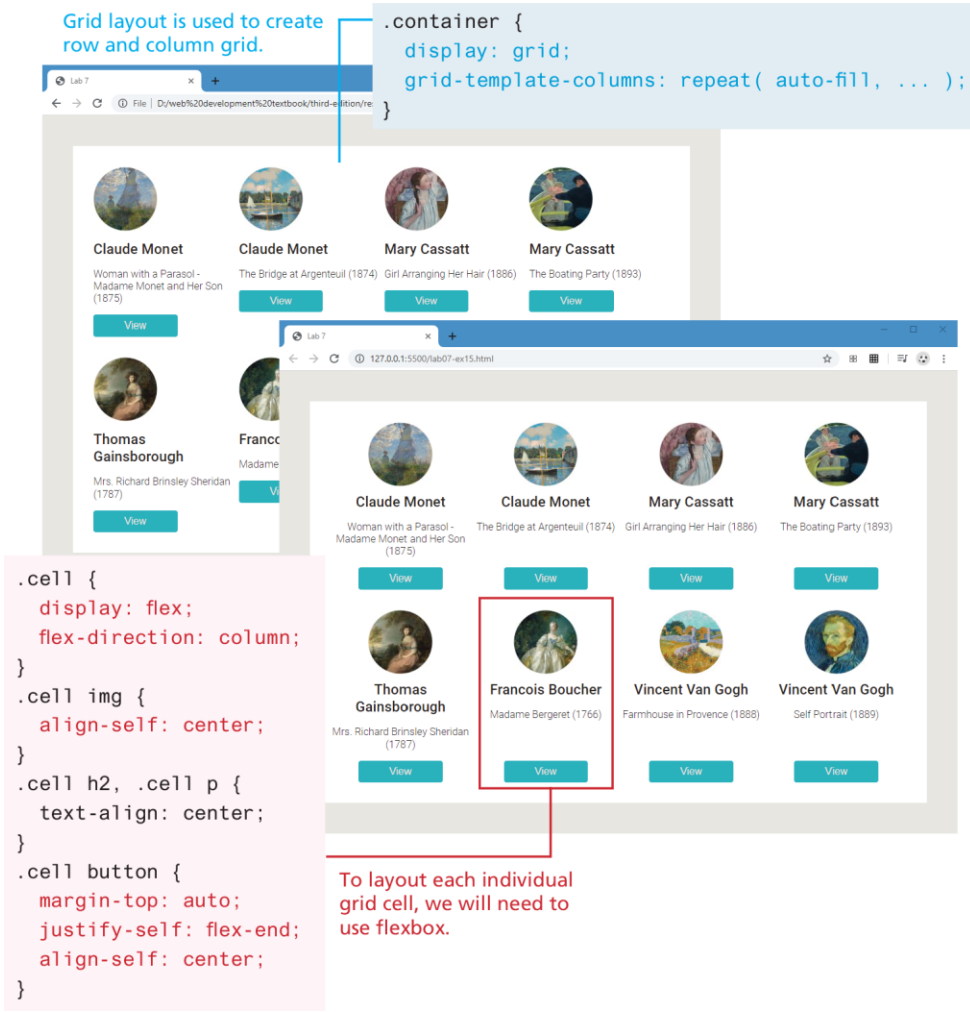
LISTING 7.2 Using grid areas

Pearson

Copyright © 2021, 2018, 2015 Pearson Education, Inc. All Rights Reserved

Grid and Flexbox Together

- **grid** and **flexbox** each have their strengths and these strengths can be combined
- **grid** layout is ideal for constructing the layout structure of your page
- **flexbox** is ideal for laying out the contents of a grid cell.



Pearson

Copyright © 2021, 2018, 2015 Pearson Education, Inc. All Rights Reserved

Section 5 of 7

APPROACHES TO CSS LAYOUT

Approaches to CSS Layout

One of the main problems faced by web designers is that the size of the screen used to view the page can vary quite a bit.

- 21-inch wide screen monitor that can display 1920x1080 pixels
- older iPhone with a 3.5 screen and a resolution of 320x480 pixels

Satisfying both users can be difficult; the approach to take for one type of site content might not work as well with another site with different content.

Approaches to CSS Layout

Most designers take one of two basic approaches to dealing with the problems of screen size.

- Fixed Layout
- Liquid Layout
- Hybrid Layout

Fixed Layout

It isn't even broken

In a **fixed layout**, the basic width of the design is set by the designer, typically corresponding to an "ideal" width based on a "typical" monitor resolution

The advantage of a fixed layout is that it is easier to produce and generally has a predictable visual result.

Fixed layouts have drawbacks.

- For larger screens, there may be an excessive amount of blank space to the left and/or right of the content.
- It is also optimized for typical desktop monitors; however, as more and more user visits are happening via smaller mobile devices

Fixed Layout

Notice the fixed size



960px

Extra space to right

```
div#wrapper {
  width: 960px;
  background-color: tan;
}
```

```
<body>
   <div id="wrapper">
      <header>
      ...
      </header>
      <div id="main">
      ...
      </div>
      <footer>
      ...
      </footer>
   </div>
</body>
```



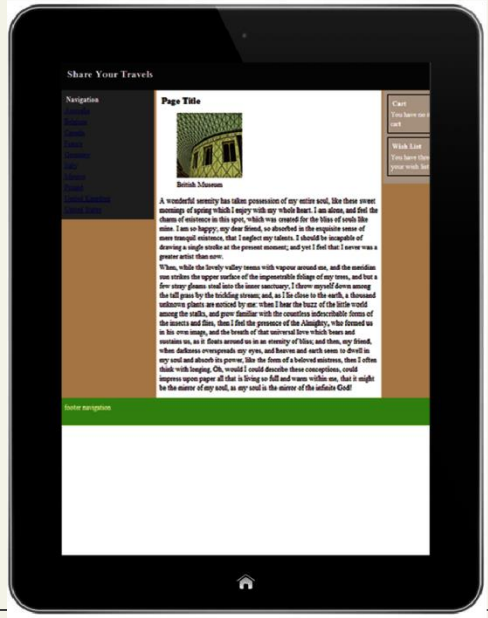
960px

Equal space to the left and to right

```
div#wrapper {
  width: 960px;
  margin-left: auto;
  margin-right: auto;
  background-color: tan;
}
```

Problem with fixed layouts





The problem with fixed layouts is that they don't adapt to smaller viewports.

Liquid Layout

Liquidy goodness

In a **liquid layout** (also called a **fluid layout**) widths are not specified using pixels, but percentage values.

Advantage:

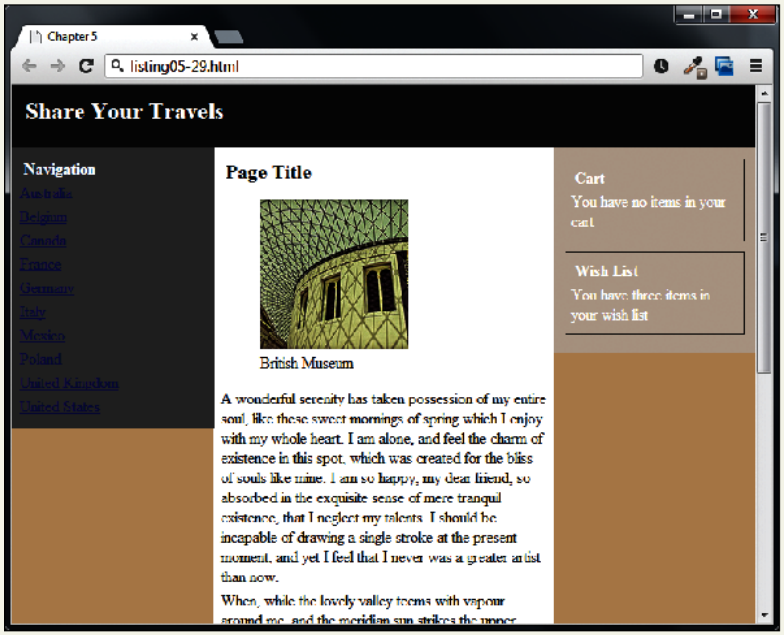
- adapts to different browser sizes,

Disadvantages:

- Liquid layouts can be more difficult to create because some elements, such as images, have fixed pixel sizes
- the line length (which is an important contributing factor to readability) may become too long or too short

Liquid Layout

Liquidy goodness



Fluid layouts are based on the browser window.

However, elements can get too spread out as browser expands.



Hybrid Layout

Such a smug feeling

A **hybrid layout** combines pixel and percentage measurements.

- Fixed pixel measurements might make sense for a sidebar column containing mainly graphic advertising images that must always be displayed and which always are the same width.
- Percentages would make more sense for the main content or navigation areas, with perhaps min and max size limits in pixels set for the navigation areas.

Section 6 of 7

RESPONSIVE DESIGN

Responsive Design

In a **responsive design**, the page "responds" to changes in the browser size that go beyond the width scaling of a liquid layout.

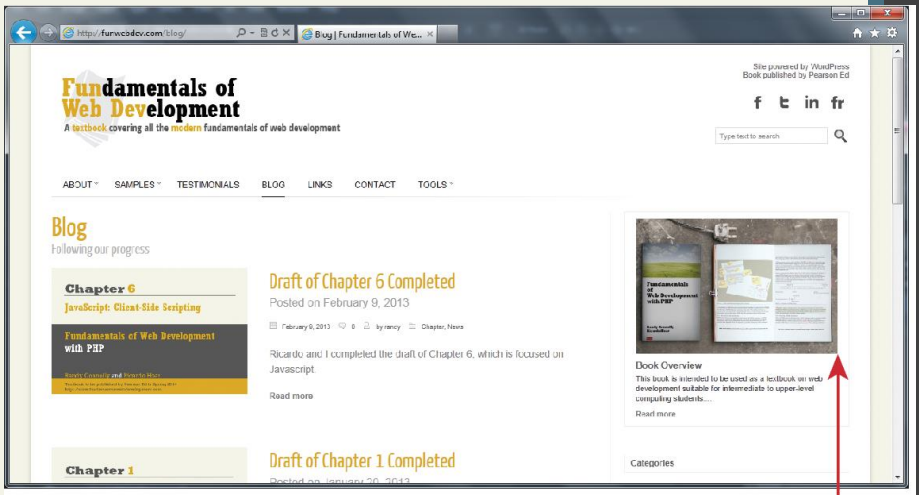
One of the problems of a liquid layout is that images and horizontal navigation elements tend to take up a fixed size, and when the browser window shrinks to the size of a mobile browser, liquid layouts can become unusable.

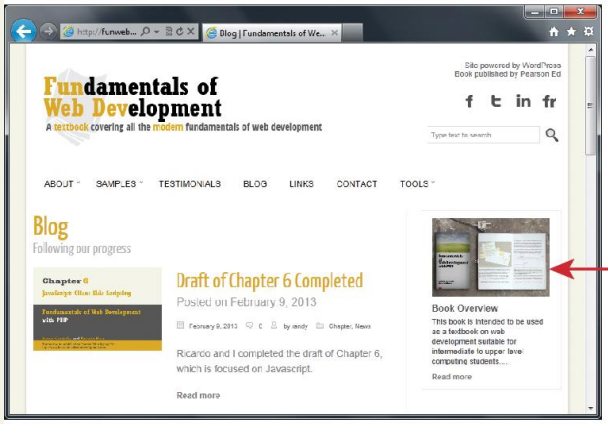
In a responsive layout, images will be scaled down and navigation elements will be replaced as the browser shrinks.

Responsive Design

In a **responsive design**, the page "responds" to changes in the browser size that go beyond simple percentage scaling of widths.

- smaller images will be served and
- navigation elements will be replaced as the browser window shrinks





Notice how some elements are scaled to shrink as browser window reduces in size.



When browser shrinks below a certain threshold, then layout and navigation elements change as well.

In this case, the `<ul>` list of hyperlinks changes to a `<select>` and the two-column design changes to one column.

Responsive Design

Example

There are four key components that make responsive design work.

1. Liquid layouts
2. Scaling images to the viewport size
3. Setting viewports via the <meta> tag
4. Customizing the CSS for different viewports using media queries

Responsive Design

Liquid Layout

Responsive designs begin with a liquid layout, that is, one in which most elements have their widths specified as percentages. Making images scale in size is actually quite straightforward, in that you simply need to specify the following rule:

```
img {

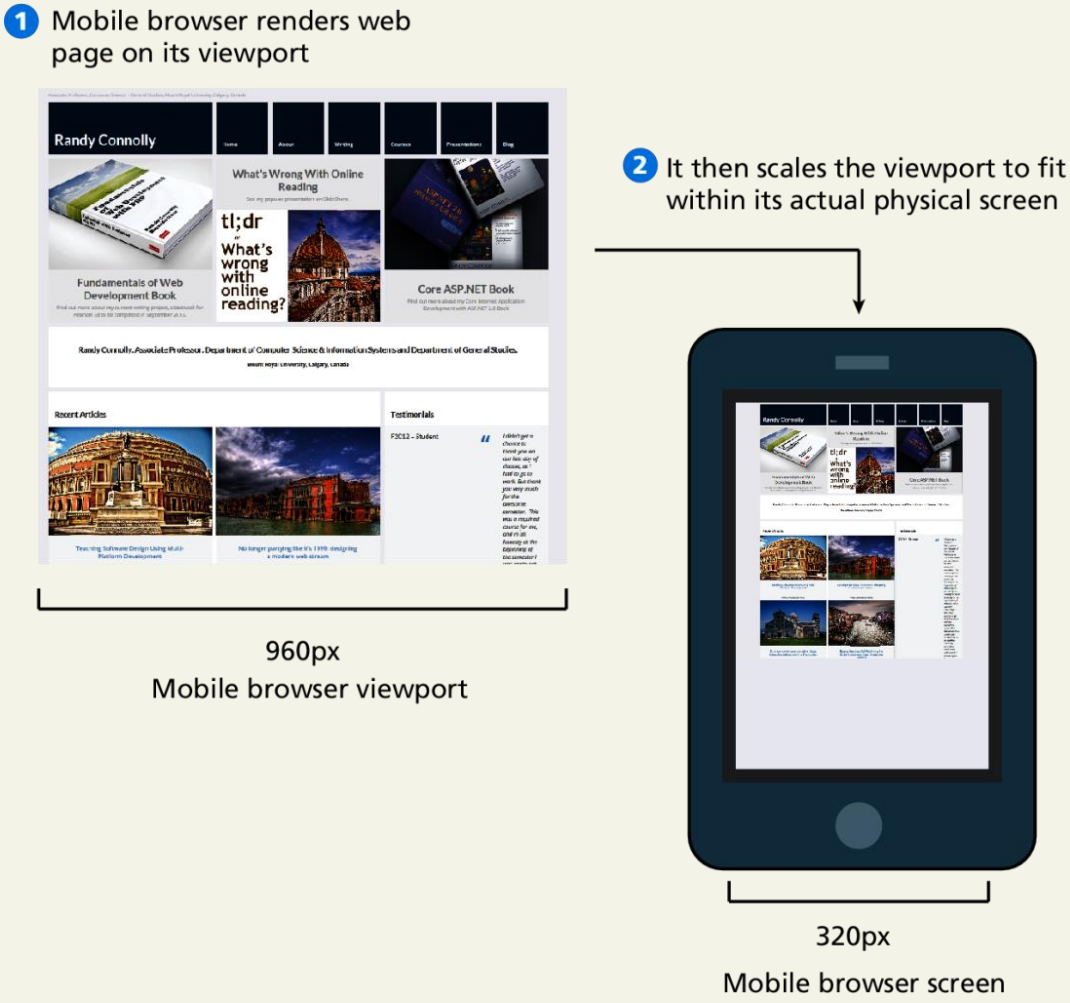
        max-width: 100%;

}
```

Responsive Design

Setting Viewports

A key technique in creating responsive layouts makes use of the ability of current mobile browsers to shrink or grow the web page to fit the width of the screen.



Responsive Design

Setting Viewports

If the developer has created a responsive site that will scale to fit a smaller screen, she may not want the mobile browser to render it on the full-size viewport. The web page can tell the mobile browser the viewport size to use via the viewport <meta> element

```
<html>

<head>

<meta name="viewport" content="width=device-width" >
```

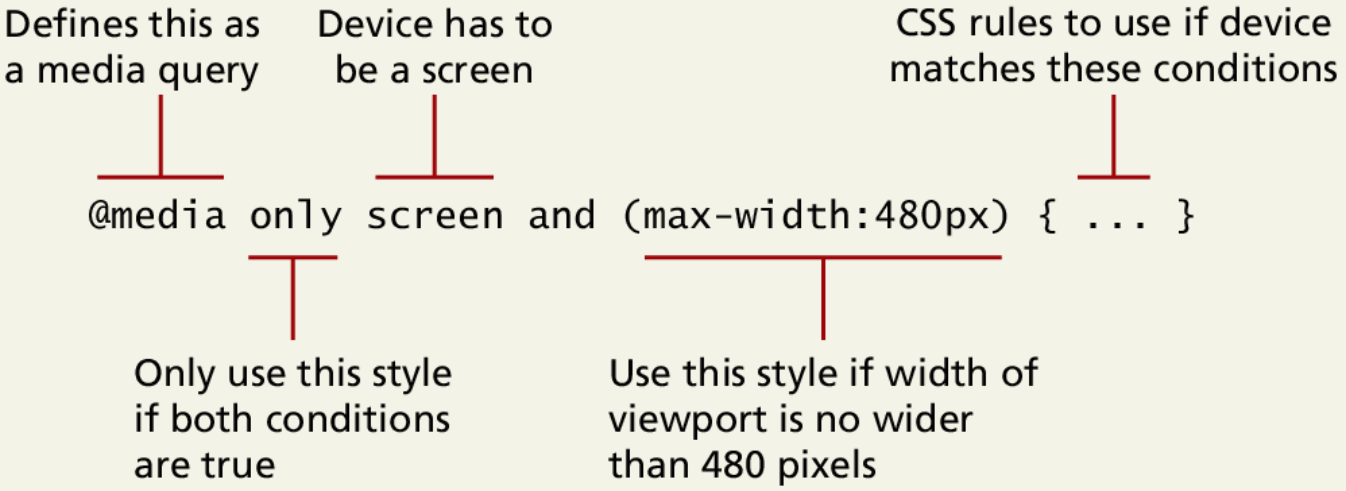
Responsive Design

Setting Viewports



Media Queries

The other key component of responsive designs is **CSS media queries**. A media query is a way to apply style rules based on the medium that is displaying the file. You can use these queries to look at the capabilities of the device, and then define CSS rules to target that device.



Media Queries

Contemporary responsive sites will typically provide CSS rules for phone displays first, then tablets, then desktop monitors, an approach called **progressive enhancement**, in which a design is adapted to progressively more advanced devices

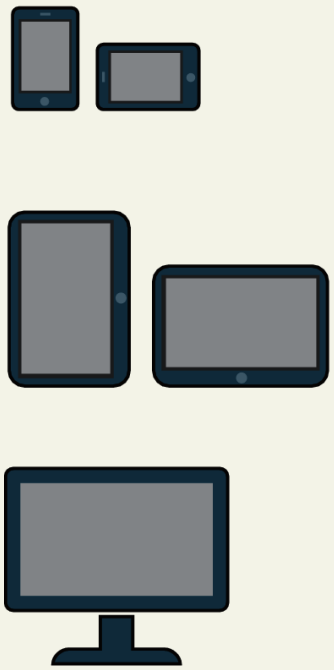
Media Queries

Browser features you can Examine with Media Queries

Feature	Description
width	Width of the viewport
height	Height of the viewport
device-width	Width of the device
device-height	Height of the device
orientation	Whether the device is portrait or landscape
color	The number of bits per color

Media Queries

Media queries in action



styles.css

```
/* rules for phones */
@media only screen and (max-width:480px)
{
  #slider-image { max-width: 100%; }
  #flash-ad { display: none; }
   ...
}


/* CSS rules for tablets */
@media only screen and (min-width: 481px)
    and (max-width: 768px)
{
   ...
}



/* CSS rules for desktops */
@media only screen and (min-width: 769px)
{
   ...
}
```

Instead of having all the rules in a single file, we can put them in separate files and add media queries to <link> elements.

```
<link rel="stylesheet" href="mobile.css"  media="screen and (max-width:480px)" />
<link rel="stylesheet" href="tablet.css"  media="screen and (min-width:481px)
    and (max-width:768px)" />
<link rel="stylesheet" href="desktop.css" media="screen and (min-width:769px)" />
```

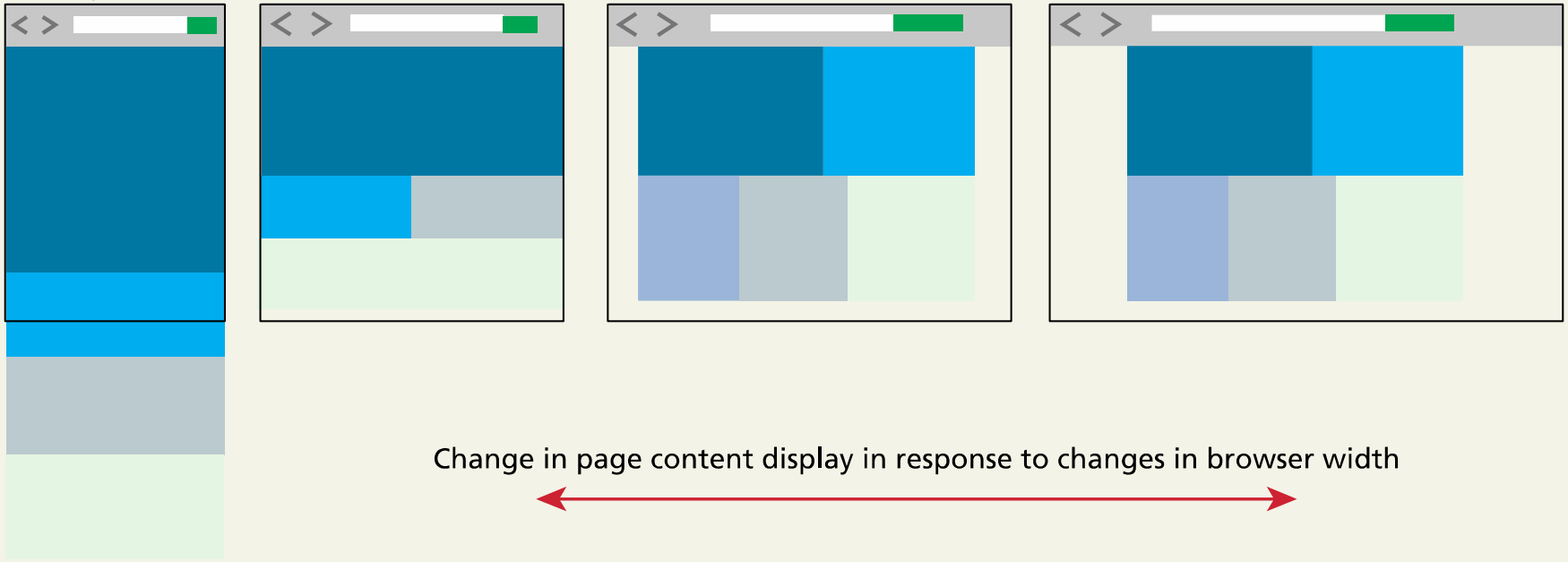
```
<!--[if lt IE 9]>
<link rel="stylesheet" media="all" href="style-ie.css"/>
<![endif]-->
```

Handles Internet Explorer 8 and earlier using IE conditional comments.

Responsive Design

Responsive Design Patterns

Mostly Fluid

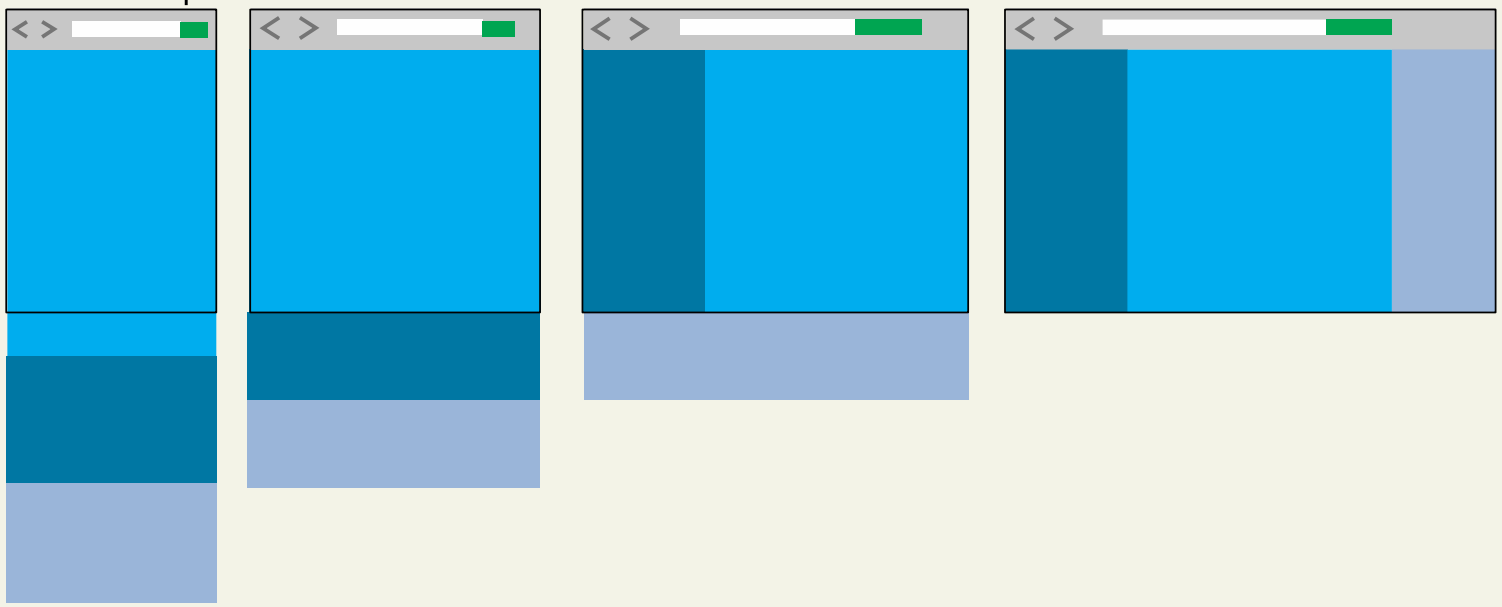


Change in page content display in response to changes in browser width

Responsive Design

Responsive Design Patterns

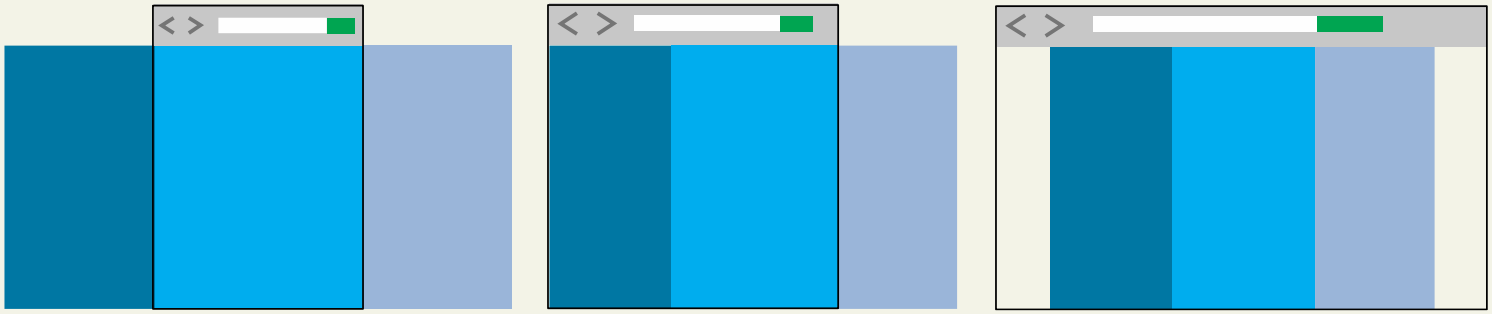
Column Drop



Responsive Design

Responsive Design Patterns

Off Canvas



Scaling images

Making images scale in size is straightforward, but does not change the downloaded size of the image

```
img {
    max-width: 100%;
}
```

HTML5.1 defines the new <picture> element that lets the designer specify multiple <img> elements. The browser determines which <img> to use based on the viewport size.

Responsive Design

Scaling Images

```
<picture>
    <source media="(min-width:960px)"
            srcset="images/828-large.jpg">
    <source media="(min-width:480px)"
            srcset="images/828-medium.jpg">
    <img alt="..." src="images/828-small.jpg">
</picture>
```

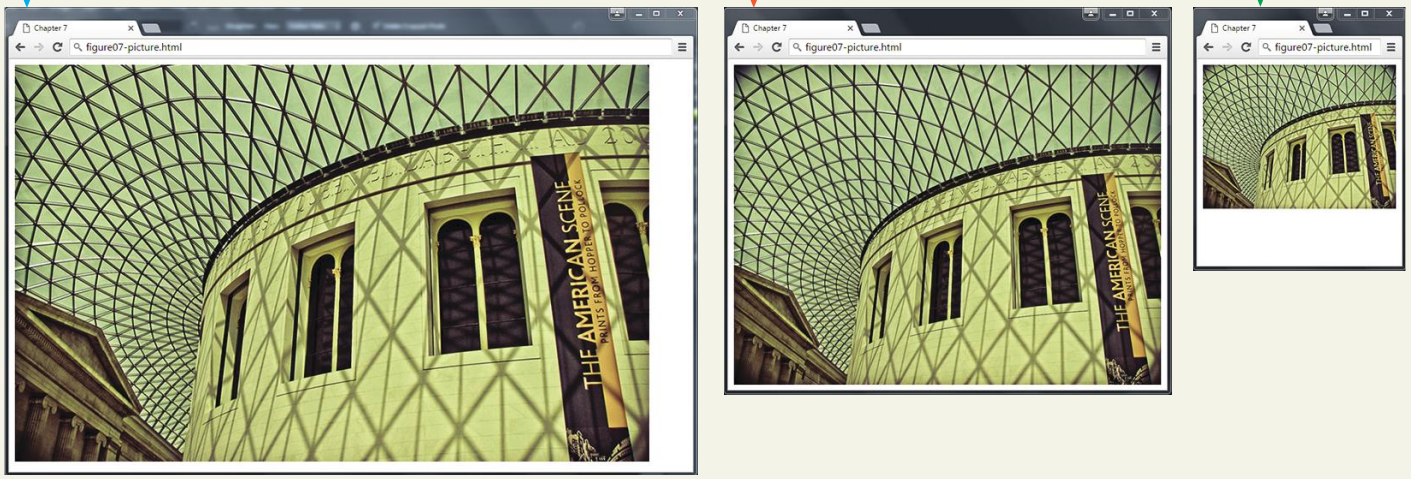
Is this true?

if yes, then use this as the src for the <img>

Is this true?

if yes, then use this as the src for the <img>

Otherwise use the src specified in the <img>



Grid Systems

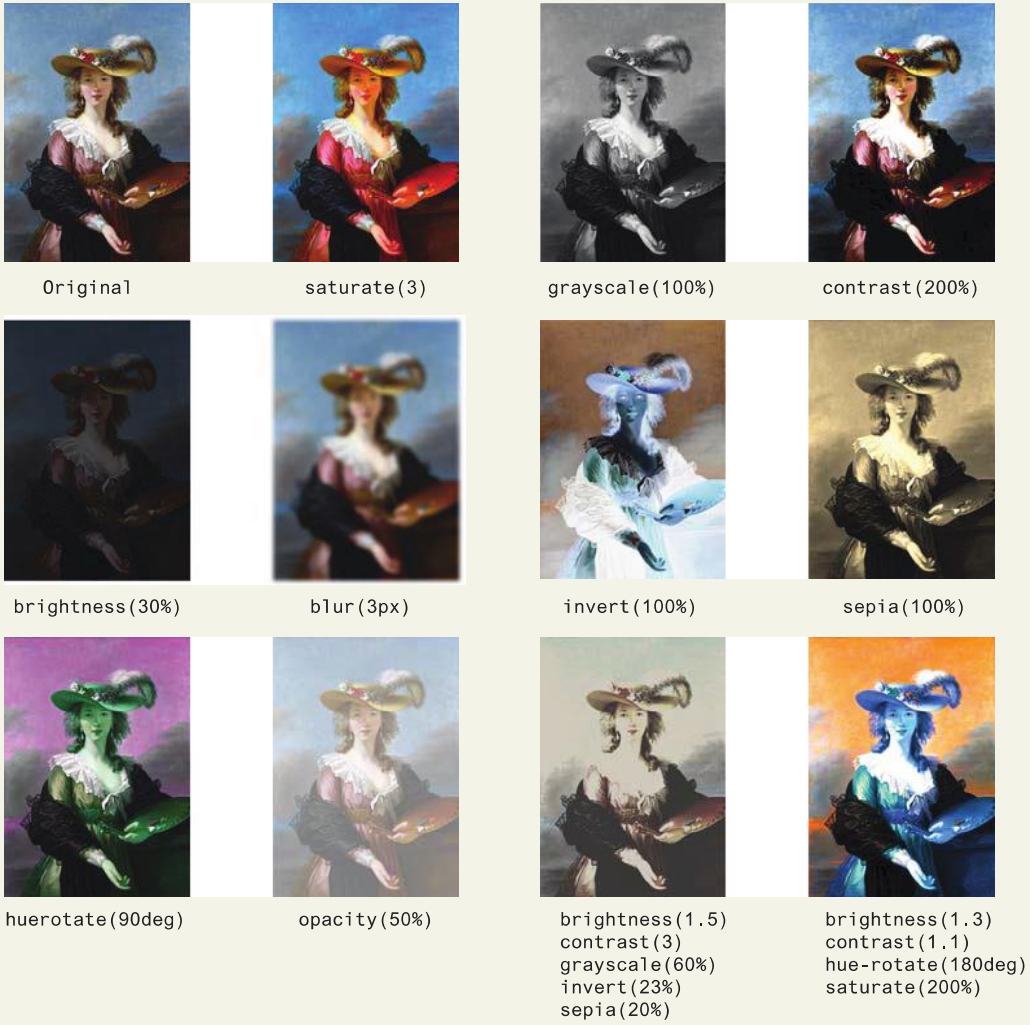
Bootstrap code

```
<head>
   <link href="bootstrap.css" rel="stylesheet">
</head>
<body>
  <div class="container">
    <div class="row">
      <div class="col-md-2">
        left column
      </div>
      <div class="col-md-7">
         main content
      </div>
      <div class="col-md-3">
        right column
      </div>
    </div>
  </div>
</body>
```

LISTING 5.3 Using the Bootstrap grid

Filters, Transitions, and Animations

Filters



Filters, Transitions, and Animations

Filters

```
#someImage {
        filter: grayscale(100%);
}

#anotherImage {
        /* multiple filters are space separated */
        filter: blur(5px) hue-rotate(60deg) saturate(2);
}
```

Transitions

Transitions are a powerful new feature of CSS3. **Transitions** will take effect across a length of time. provide a way to indicate that a property change

- **transition-delay** The delay time in seconds before the animation begins.
- **transition-duration** How long in seconds for the transition to complete.
- **transition-property** The name of the CSS property to which the transition is applied.
- **Transition-timing-function** The function that defines how the intermediate steps in the transition are calculated. CSS defines a variety of different easing functions which define the rate of the transition.

Filters, Transitions, and Animations

Transitions

The button as it normally appears

Button as it appears during transition between two states

The button as it appears when hovered over



This dark green color will be displayed when the mouse is not over the button.

```
button {
    background-color: #146d37;
```

Which CSS property of the button is going to be transitioned across time?

1. `transition-property: background-color;`

We will transition the background color of the button across time.

2. How long is the transition?

`transition-duration: 0.5s;`

The transition will last half a second.

3. What will be the rate transition change?

`transition-timing-function: ease-out;`

The transition will slow down towards the end.

4. Do we delay the start of the transition?

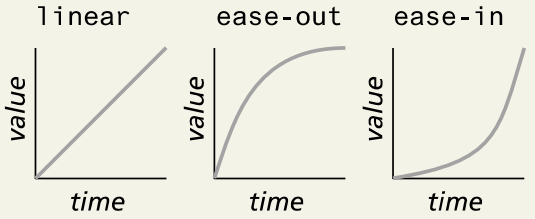
`transition-delay: 0s;`

No delay (transition will start immediately)

```
}
```

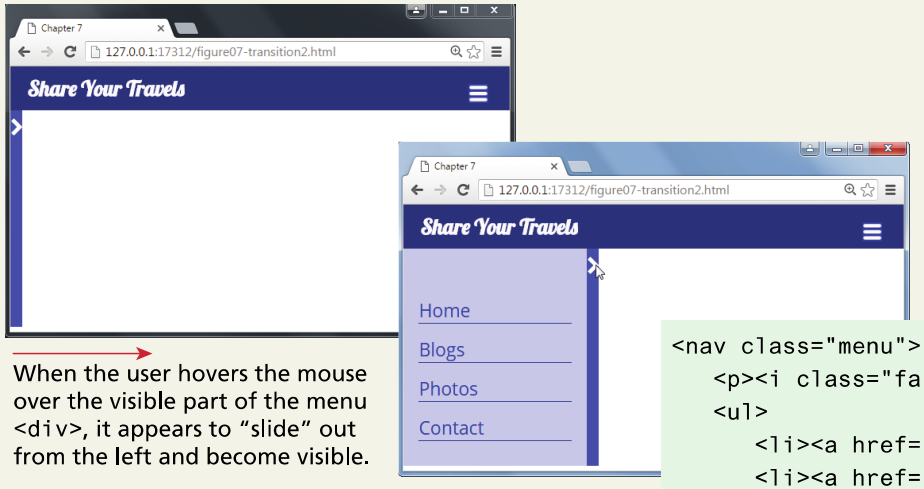
```
button:hover {
    background-color: #60b946;
}
```

This light green color will be displayed when the mouse hovers over the button.



Filters, Transitions, and Animations

Transitions



When the user hovers the mouse over the visible part of the menu , it appears to "slide" out from the left and become visible.

```
<nav class="menu">
   <p><i class="fa fa-chevron-right"></i></p>
   <ul>
      <li><a href="#">Home</a></li>
      <li><a href="#">Blogs</a></li>
      <li><a href="#">Photos</a></li>
      <li><a href="#">Contact</a></li>
   </ul>
</nav>
```

Menu is initially hidden by being positioned outside the visible area

```
.menu {
   position: absolute;
   left: -210px;
}

.menu:hover {
   left: 0;

   transition: left .6s ease-out;
}
```

When the user hovers over the menu, move the left edge of the element to left edge of the browser (i.e., it will now be visible).

Using the transition shorthand property

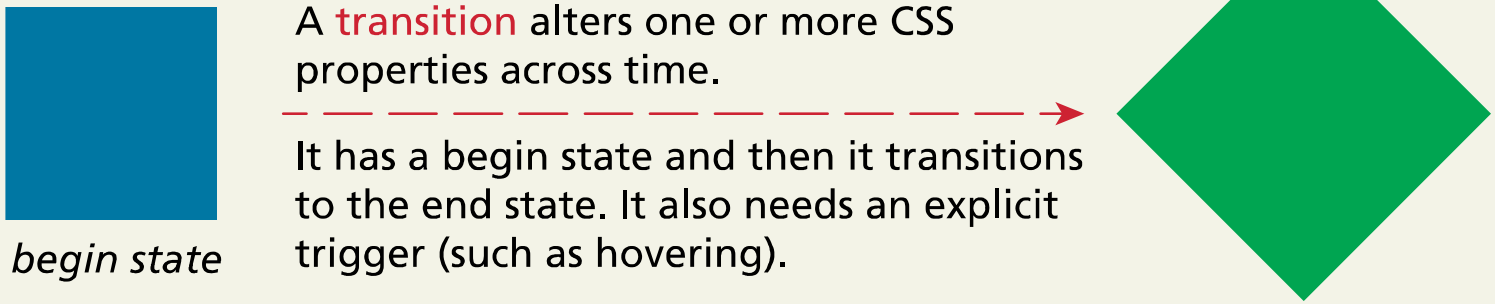
Transition the `left` property across 0.6 seconds and use the ease-out function (i.e., slow down transition at end)

```
.menu {
   transition: left .6s ease-out;
}
```

We want the same transition when the mouse is no longer hovering over the menu. This creates illusion of menu sliding back out of sight.

Filters, Transitions, and Animations

Transitions vs animations

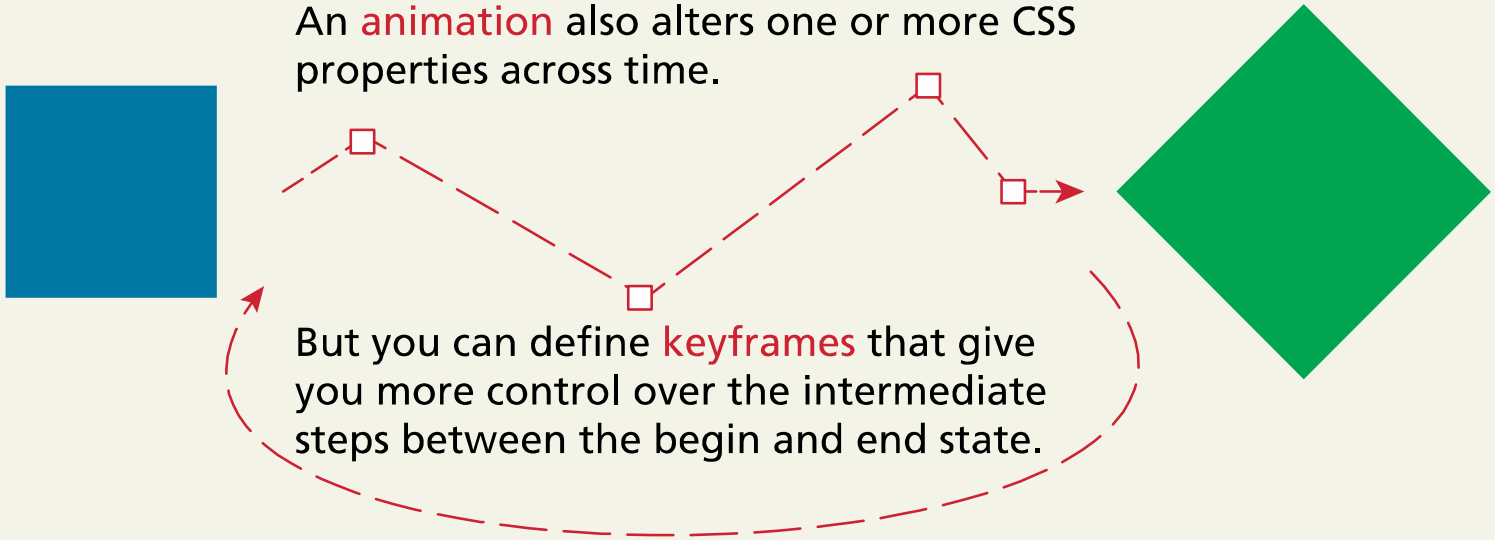


begin state

A transition alters one or more CSS properties across time.

It has a begin state and then it transitions to the end state. It also needs an explicit trigger (such as hovering).

end state



An animation also alters one or more CSS properties across time.

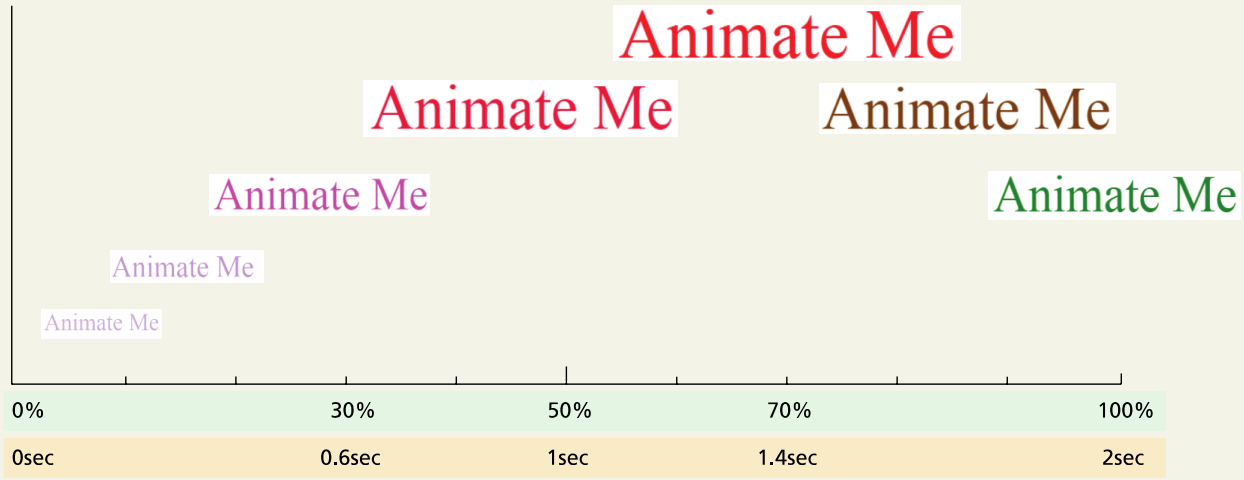
But you can define keyframes that give you more control over the intermediate steps between the begin and end state.

No trigger is needed: an animation begins once it is defined. As well, you can also loop an animation.

Filters, Transitions, and Animations

Animations

```
<p class="animated">Animate Me</p>
```




```
.animated {
   animation-iteration-count: infinite;
   animation-name: bounceIn;
   animation-play-state: running;
   animation-duration: 2s;
   animation-timing-function: ease-out;
   animation-delay: 1s;
}

.animated:hover {
   animation-play-state: paused;
}
```

- Run animation indefinitely
- Play animation named bounceIn
- Play animation once it is defined
- Animation lasts 2 seconds
- Slow animation towards the end
- Wait a second before starting animation

- Pause the animation by hovering over it (useful for debugging only)